

浙江工业大学

计算机网络课程设计报告



学 号 XXXXXXXXXXXXXX

姓 名 XXX

班 级 XX 学院 XX 专业 XX 班

提交日期 20XX 年 X 月

目 录

一、 课程的目的和任务	2
1.1 课程目标	2
1.2 课程思政	2
1.3 课程任务	2
二、 课程的基本内容和要求	4
2.1 常用网络命令	4
2.1.1 课程要求	4
2.1.2 ipconfig.....	4
2.1.3 ping	5
2.1.4 netstat	7
2.1.5 tracert.....	10
2.1.6 arp.....	11
2.1.7 telnet	13
2.2 交换机与路由器	15
2.2.1 课程要求	15
2.2.2 交换机命令学习及交换机初始化设置	15
2.2.3 交换机及 VLAN 配置	17
2.2.4 路由器命令学习及路由器初始化设置	21
2.2.5 静态路由配置	23
2.2.6 动态路由配置 RIP	26
2.2.7 动态路由配置 OSPF	32
2.2.8 综合网络情景设计	35
2.3 网络编程	40
2.3.1 课程要求	40
2.3.2 设计思想	40
2.3.3 实验环境	40
2.3.4 主要函数与结构体分析.....	41
2.3.5 程序流程图.....	44
2.3.6 难点解决	44
2.3.7 测试结果截图	45
2.3.8 实验结果分析	46
2.3.9 程序源代码.....	47
三、 课程设计总结体会	52

一、课程的目的和任务

1.1 课程目标

- (1) 理解计算机网络体系结构和工作原理，掌握常用网络命令的功能，能够分析命令执行结果，运用常用网络命令进行网络测试与故障检测，并得到合理有效的结论。
- (2) 能够进行网络组网设计，配置交换机和路由器，按照实验方案实施仿真实验，采集和整理实验数据，并对实验数据进行分析、处理和解释。
- (3) 具备网络编程能力，能够进行简单网络编程，掌握设计协议分析程序的方法。
- (4) 具备自主学习和创新意识，具有不断学习和适应技术发展的能力。

1.2 课程思政

- (1) 构建方案应立足实践基础，勇于创新 and 尝试，通过不断优化与迭代，实现精益求精。
- (2) 尊重科学，实事求是。对于所设计的方案，能够通过实验进行验证，注重理论与实践相结合。
- (3) 强化团结协作意识和责任担当精神，增强责任心与使命感。

1.3 课程任务

任务一、运用网络命令进行网络故障检测分析。掌握计算机网络相关命令原理及应用。常用网络命令 `ipconfig`, `ping`, `netstat`, `tracert`, `arp`, `telnet` 的功能；在 windows 环境下使用上述网络命令进行网络状态监测和跟踪，给出相应的截图和对结果的解释。

任务二、基于交换机命令学习及交换机初始化设置的特定任务需求，构建系统方案、实施实验、处理数据并分析实验结果。基于交换机 VLAN 设置的特定任务需求，构建系统方案、实施实验、处理数据并分析实验结果。基于路由器端口、路由器命令学习和路由器登录方法的特定任务需求，构建系统方案、实施实验、并验证。基于静态路由、动态路由配置的特定任务需求，构建系统方案、实施实验、处理数据并分析实验结果。

(1) 安装 packet tracer，在 packet tracer 仿真环境下，熟悉交换机命令、交换机初始化配置；

(2) 在交换机上实现 VLAN 配置；

要求：创建三个 VLAN，给出拓扑，查看 VLAN 信息

(3) 基于 Console 控制台登录配置路由器，学习路由器配置相关命令；

(4) 基于 packet tracer 构建网络环境，分别进行静态路由配置和基于 RIP 的动态路由配置，设计基于 OSPF 的动态路由配置。能够进行综合集成网络情景设计更佳。

任务三、基于网络编程的特定任务需求进行 socket 编程，设计处理流程、数据结构、实施实验并分析实验结果。编程要求：捕获本机网卡的 IP 包，对捕获的 IP 包进行解析。要求必须输出以下字段：版本号、总长度、标志位、片偏移、协议、源地址和目的地址。

要求有详细的说明文档，包括程序的设计思想、工作流程、关键问题、程序注释和对捕获包的解析截图。

编程语言不作要求，可使用自己熟悉的 C、C++、java 或 C 等。

二、课程的基本内容和要求

2.1 常用网络命令

2.1.1 课程要求

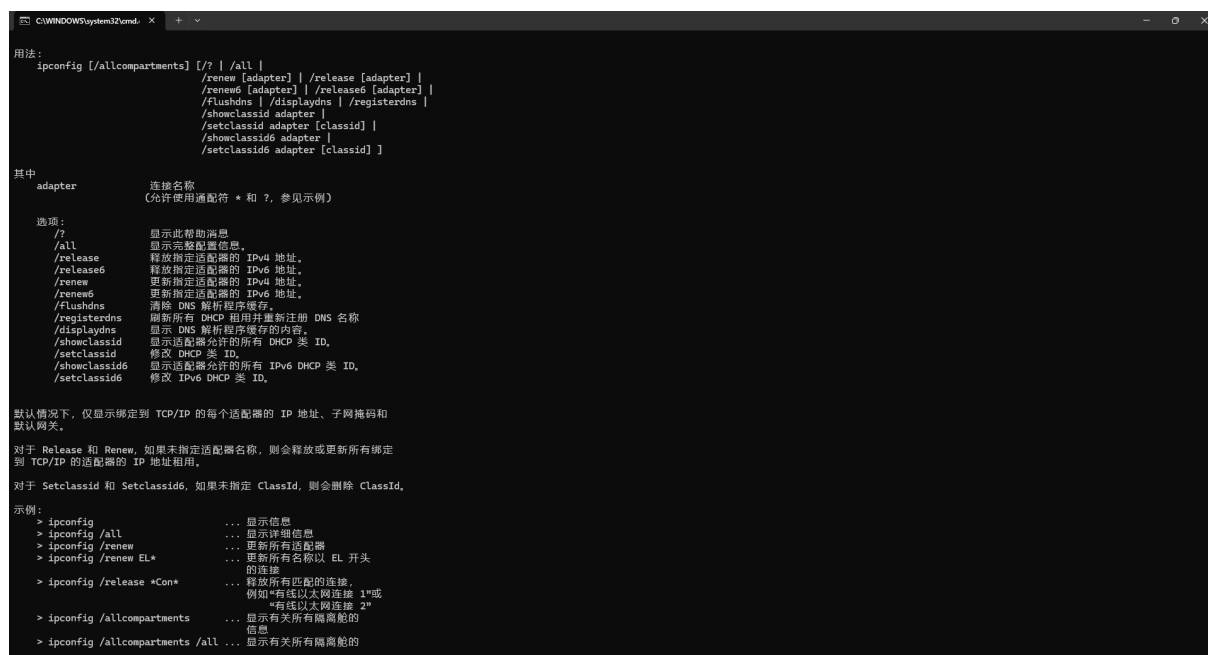
运用网络命令进行网络故障检测分析。掌握计算机网络相关命令原理及应用。常用网络命令 ipconfig, ping, netstat, tracert, arp, telnet 的功能；在 windows 环境下使用上述网络命令进行网络状态监测和跟踪，给出相应的截图和对结果的解释。

2.1.2 ipconfig

功能、参数及原理

该命令可以查看本机网络适配器的相关信息，包括各网络适配器的型号、MAC 地址、IP 地址、子网掩码、默认网关及 IP 租用时间。该命令还可以用于查看和刷新 DNS 缓存，以及用于进行 DHCP 相关的配置。

通过执行 ipconfig /?，可以查看该命令的参数。



```
用法:
ipconfig [/allcompartments] [/? | /all |
        /renew [adapter] | /release [adapter] |
        /renew6 [adapter] | /release6 [adapter] |
        /flushdns | /displaydns | /registerdns |
        /showclassid adapter |
        /setclassid adapter [classid] |
        /showclassid6 adapter |
        /setclassid6 adapter [classid] ]

其中
adapter          连接名称
                  (允许使用通配符 * 和 ?, 参见示例)

选项:
/?              显示此帮助消息。
/all           显示完整配置信息。
/release       释放指定适配器的 IPv4 地址。
/release6      释放指定适配器的 IPv6 地址。
/renew         更新指定适配器的 IPv4 地址。
/renew6        更新指定适配器的 IPv6 地址。
/flushdns      清除 DNS 解析程序缓存。
/registerdns   刷新所有 DHCP 租约并重新注册 DNS 名称。
/displaydns    显示 DNS 解析程序缓存的内容。
/showclassid   显示适配器允许的所有 DHCP 类 ID。
/setclassid    修改 DHCP 类 ID。
/showclassid6  显示适配器允许的所有 IPv6 DHCP 类 ID。
/setclassid6   修改 IPv6 DHCP 类 ID。

默认情况下，仅显示绑定到 TCP/IP 的每个适配器的 IP 地址、子网掩码和默认网关。

对于 Release 和 Renew，如果未指定适配器名称，则会释放或更新所有绑定到 TCP/IP 的适配器的 IP 地址租用。

对于 Setclassid 和 Setclassid6，如果未指定 ClassId，则会删除 ClassId。

示例:
> ipconfig          ... 显示信息
> ipconfig /all     ... 显示详细信息
> ipconfig /renew    ... 更新所有适配器
> ipconfig /renew EL* ... 更新所有名称以 EL 开头的连接
> ipconfig /release *Con* ... 释放所有匹配的连接。
                        例如“有线以太网连接 1”或
                        “有线以太网连接 2”
> ipconfig /allcompartments ... 显示有关所有隔离舱的信息
> ipconfig /allcompartments /all ... 显示有关所有隔离舱的信息
```

图 1: ipconfig /? 命令执行结果节选

应用与分析

接下来，我们将应用两个常见的命令，对其运行结果进行分析。

下图为执行 ipconfig /all 所得结果的节选，它描述了该本机无线网络适配器的配置信息。

```
无线局域网适配器 WLAN:
    连接特定的 DNS 后缀 . . . . . : 
    描述 . . . . . : Intel(R) Wi-Fi 6E AX211 160MHz
    物理地址. . . . . : E8-BF-B8-5F-A0-50
    DHCP 已启用 . . . . . : 是
    自动配置已启用 . . . . . : 是
    本地链接 IPv6 地址. . . . . : fe80::7f19:9b53:99c4:108c%17(首选)
    IPv4 地址 . . . . . : 10.136.26.146(首选)
    子网掩码 . . . . . : 255.255.240.0
    获得租约的时间 . . . . . : 2026年1月6日 20:24:34
    租约过期的时间 . . . . . : 2026年1月10日 11:05:11
    默认网关 . . . . . : 10.136.16.1
    DHCP 服务器 . . . . . : 10.136.16.1
    DHCPv6 IAID . . . . . : 149471160
    DHCPv6 客户端 DUID . . . . . : 00-01-00-01-2E-1F-34-9B-28-C5-C8-D2-92-BB
    DNS 服务器 . . . . . : 172.16.7.10
                           172.16.7.30
    TCP/IP 上的 NetBIOS . . . . . : 已启用
```

图 2: ipconfig /all 命令执行结果节选

- 描述：该网络适配器的型号为 Intel(R) Wi-Fi 6E AX211 160MHz。
- 物理地址：该网络适配器的 MAC 地址为 E8-BF-B8-5F-A0-50。
- DHCP 已启用：该网络适配器已启用 DHCP，意味着允许使用 DHCP 自动获取 IP 地址。
- IPv4 地址：该网络适配器的首选 IPv4 地址为 10.136.26.146，其 IPv4 地址可以有多个，IPv6 地址同理。
- 子网掩码：该网络适配器 IPv4 地址所对应的子网掩码为 255.255.240.0。
- 获得租约的时间：和租约过期的时间表示由 DHCP 临时租用的 IP 地址的可用时间段。
- 默认网关：该网络适配器 IPv4 地址所对应的默认网关为 10.136.16.1。
- DNS 服务器：该网络适配器所使用的 DNS 服务器为 10.136.16.1，可以有多个。

下图为执行 `ipconfig /flushdns` 所得的结果。通过这个命令，我们清除了本机 DNS 解析程序的缓存。DNS 缓存通过对一些 DNS 记录进行缓存，从而加速 DNS 查询的效率，我们还可以执行 `ipconfig /displaydns` 来查看其缓存内容。

```
C:\Users\19711>ipconfig /flushdns

Windows IP 配置

已成功刷新 DNS 解析缓存。

C:\Users\19711>
```

图 3: ipconfig /flushdns 命令执行结果节选

2.1.3 ping

功能、参数及原理

该命令可以向指定主机发送 ICMP 请求报文，并返回数据包大小，往返时间，TTL 等，常用于检测本机与目的主机间是否可达。

通过执行 `ping /?`，可以查看该命令的参数。

```
C:\WINDOWS\system32\cmd. X + v
C:\Users\19711>ping /?

用法: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
        [-r count] [-s count] [[-j host-list] | [-k host-list]]
        [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
        [-4] [-6] target_name

选项:
    -t          Ping 指定的主机, 直到停止。
                若要查看统计信息并继续操作, 请键入 Ctrl+Break;
                若要停止, 请键入 Ctrl+C。
    -a          将地址解析为主机名。
    -n count    要发送的回显请求数。
    -l size     发送缓冲区大小。
    -f          在数据包中设置“不分段”标记(仅适用于 IPv4)。
    -i TTL      生存时间。
    -v TOS      服务类型(仅适用于 IPv4。该设置已被弃用,
                对 IP 标头中的服务类型字段没有任何影响)。
    -r count    记录计数跃点的路由(仅适用于 IPv4)。
    -s count    计数跃点的时间戳(仅适用于 IPv4)。
    -j host-list 与主机列表一起使用的松散源路由(仅适用于 IPv4)。
    -k host-list 与主机列表一起使用的严格源路由(仅适用于 IPv4)。
    -w timeout  等待每次回复的超时时间(毫秒)。
    -R          同样使用路由标头测试反向路由(仅适用于 IPv6)。
                根据 RFC 5095, 已弃用此路由标头。
                如果使用此标头, 某些系统可能丢弃回显请求。
    -S srcaddr  要使用的源地址。
    -c compartment 路由隔离舱标识符。
    -p          Ping Hyper-V 网络虚拟化提供程序地址。
    -4          强制使用 IPv4。
    -6          强制使用 IPv6。

C:\Users\19711>
```

图 4: ping /? 命令执行结果节选

应用与分析

接下来, 我们加上一些参数执行 ping 命令。

如下图所示, 我们执行 `ping 10.136.26.164 -l 100 -n 4`, 向 `ipconfig` 命令查出的无线网络适配器 10.136.26.164 发送 4 个数据大小为 100 字节的数据包。

```
C:\WINDOWS\system32\cmd. X + v
C:\Users\19711>ping 10.136.26.146 -l 100 -n 4

正在 Ping 10.136.26.146 具有 100 字节的数据:
来自 10.136.26.146 的回复: 字节=100 时间<1ms TTL=128
来自 10.136.26.146 的回复: 字节=100 时间<1ms TTL=128
来自 10.136.26.146 的回复: 字节=100 时间<1ms TTL=128
来自 10.136.26.146 的回复: 字节=100 时间<1ms TTL=128

10.136.26.146 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\19711>
```

图 5: ping 指定地址的执行结果

所得结果我们可以看到, 我们收到了来自 10.136.26.164 的回复, 耗时 0~1ms。4 个数据包都没有丢包, 丢包率为 0%, 并且得到了最小、最大、平均 RTT。

TTL 字段是指数据包的生存时间，ICMP 包在网络中没转发一次，TTL 就会减少 1，TTL 的初始值与发送数据包的操作系统有关。目的地址 10.136.26.164 为 Win10 操作系统，通过查阅资料，其发送的响应 TTL 为 128，由上图结果 TTL 为 128，我们就可以推断响应数据包没有经过路由转发。

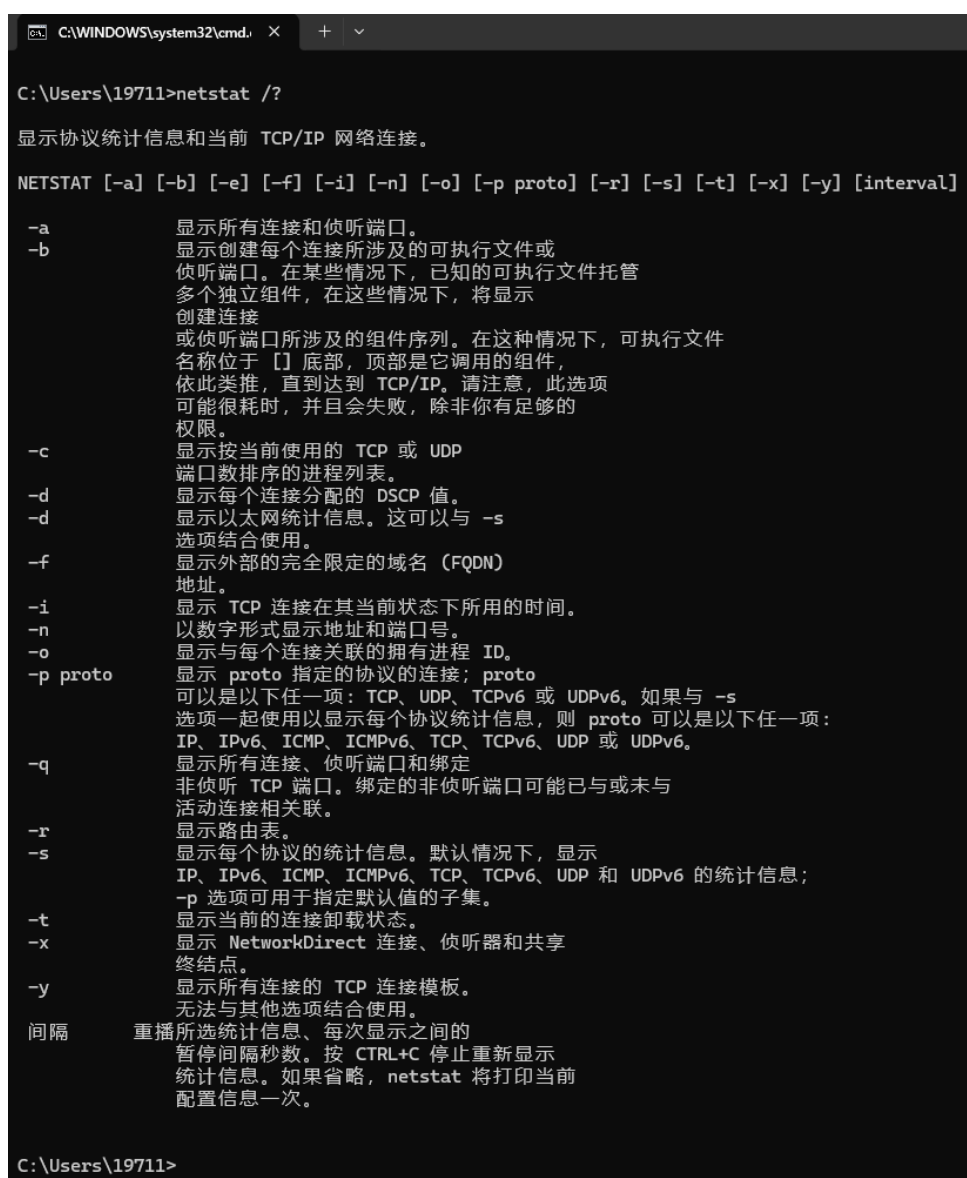
由于这个特性，在路由转发次数不是太多的情况下，我们可以用 ping 命令大致确定目的地址的操作系统。

2.1.4 netstat

功能、参数与原理

该命令可以查看在 TCP/IP 网络连接下以太网信息、各协议的统计数据、路由表等，常用来查看和监听本机各端口的网络连接状况。

通过执行 netstat /?，可以查看该命令的参数。



```
C:\WINDOWS\system32\cmd.exe
C:\Users\19711>netstat /?

显示协议统计信息和当前 TCP/IP 网络连接。

NETSTAT [-a] [-b] [-e] [-f] [-i] [-n] [-o] [-p proto] [-r] [-s] [-t] [-x] [-y] [interval]

-a          显示所有连接和侦听端口。
-b          显示创建每个连接所涉及的可执行文件或
            侦听端口。在某些情况下，已知的可执行文件托管
            多个独立组件，在这些情况下，将显示
            创建连接
            或侦听端口所涉及的组件序列。在这种情况下，可执行文件
            名称位于 [] 底部，顶部是它调用的组件，
            依此类推，直到达到 TCP/IP。请注意，此选项
            可能很耗时，并且会失败，除非你有足够的
            权限。
-c          显示按当前使用的 TCP 或 UDP
            端口数排序的进程列表。
-d          显示每个连接分配的 DSCP 值。
-d          显示以太网统计信息。这可以与 -s
            选项结合使用。
-f          显示外部的完全限定的域名 (FQDN)
            地址。
-i          显示 TCP 连接在其当前状态下所用的时间。
-n          以数字形式显示地址和端口号。
-o          显示与每个连接关联的拥有进程 ID。
-p proto    显示 proto 指定的协议的连接；proto
            可以是以下任一项：TCP、UDP、TCPv6 或 UDPv6。如果与 -s
            选项一起使用以显示每个协议统计信息，则 proto 可以是以下任一项：
            IP、IPv6、ICMP、ICMPv6、TCP、TCPv6、UDP 或 UDPv6。
-q          显示所有连接、侦听端口和绑定
            非侦听 TCP 端口。绑定的非侦听端口可能已与或未与
            活动连接相关联。
-r          显示路由表。
-s          显示每个协议的统计信息。默认情况下，显示
            IP、IPv6、ICMP、ICMPv6、TCP、TCPv6、UDP 和 UDPv6 的统计信息；
            -p 选项可用于指定默认值的子集。
-t          显示当前的连接卸载状态。
-x          显示 NetworkDirect 连接、侦听器 and 共享
            终结点。
-y          显示所有连接的 TCP 连接模板。
            无法与其他选项结合使用。
interval    重播所选统计信息、每次显示之间的
            暂停间隔秒数。按 CTRL+C 停止重新显示
            统计信息。如果省略，netstat 将打印当前
            配置信息一次。

C:\Users\19711>
```

图 6: netstat /? 执行结果

应用与分析

接下来，我们将应用两个常见的命令，对其运行结果进行分析。首先，我们用 netstat -r 查看本机路由表，运行结果如下图所示。

```

C:\WINDOWS\system32\cmd.exe
C:\Users\19711>netstat -r

=====
接口列表
12...e8 bf b8 5f a0 51 .....Microsoft Wi-Fi Direct Virtual Adapter
21...ea bf b8 5f a0 50 .....Microsoft Wi-Fi Direct Virtual Adapter #2
22...28 c5 c8 d2 92 bb .....Realtek Gaming GbE Family Controller
5...00 ff 4f ff 3e 4e .....Netease UU TAP-Win32 Adapter V9.21
11...00 50 56 c0 00 01 .....VMware Virtual Ethernet Adapter for VMnet1
7...00 50 56 c0 00 08 .....VMware Virtual Ethernet Adapter for VMnet8
56.....Netkeeper
17...e8 bf b8 5f a0 50 .....Intel(R) Wi-Fi 6E AX211 160MHz
1.....Software Loopback Interface 1
=====

IPv4 路由表
=====
活动路由:
网络目标      网络掩码      网关      接口      跃点数
0.0.0.0        0.0.0.0        172.22.0.1  172.22.41.233  4260
0.0.0.0        0.0.0.0        10.136.16.1  10.136.26.146  4275
0.0.0.0        0.0.0.0        在链路上    115.200.15.220  36
10.136.16.0    255.255.240.0  在链路上    10.136.26.146  4531
10.136.26.146  255.255.255.255  在链路上    10.136.26.146  4531
10.136.31.255  255.255.255.255  在链路上    10.136.26.146  4531
115.200.15.220  255.255.255.255  在链路上    115.200.15.220  291
127.0.0.0      255.0.0.0      在链路上    127.0.0.1      4556
127.0.0.1      255.255.255.255  在链路上    127.0.0.1      4556
127.255.255.255  255.255.255.255  在链路上    127.0.0.1      4556
172.22.0.0      255.255.0.0      在链路上    172.22.41.233  4516
172.22.41.233  255.255.255.255  在链路上    172.22.41.233  4516
172.22.255.255  255.255.255.255  在链路上    172.22.41.233  4516
192.168.142.0  255.255.255.0    在链路上    192.168.142.1  4516
192.168.142.1  255.255.255.255  在链路上    192.168.142.1  4516
192.168.142.255  255.255.255.255  在链路上    192.168.142.1  4516
192.168.233.0  255.255.255.0    在链路上    192.168.233.1  4516
192.168.233.1  255.255.255.255  在链路上    192.168.233.1  4516
192.168.233.255  255.255.255.255  在链路上    192.168.233.1  4516
224.0.0.0      240.0.0.0      在链路上    127.0.0.1      4556
224.0.0.0      240.0.0.0      在链路上    172.22.41.233  4516
224.0.0.0      240.0.0.0      在链路上    10.136.26.146  4531
224.0.0.0      240.0.0.0      在链路上    192.168.233.1  4516
224.0.0.0      240.0.0.0      在链路上    192.168.142.1  4516
224.0.0.0      240.0.0.0      在链路上    115.200.15.220  36
255.255.255.255  255.255.255.255  在链路上    127.0.0.1      4556
255.255.255.255  255.255.255.255  在链路上    172.22.41.233  4516
255.255.255.255  255.255.255.255  在链路上    10.136.26.146  4531
255.255.255.255  255.255.255.255  在链路上    192.168.233.1  4516
255.255.255.255  255.255.255.255  在链路上    192.168.142.1  4516
255.255.255.255  255.255.255.255  在链路上    115.200.15.220  291
=====
永久路由:
无
  
```

图 7: netstat -r 执行结果片段

我们来分析一下路由表的各个字段：

- **网络目标**：即目的 IP 地址。结合本机路由表可见，存在默认路由（0.0.0.0），本机回环网段（127.0.0.0），受限广播地址（255.255.255.255）以及组播网段（224.0.0.0）。同时，本机还包含多个直连网络段，例如 10.136.16.0/20、192.168.142.0/24 与 192.168.233.0/24 等。
- **网络掩码**：用于与网络目标进行按位与运算，从而判断目的 IP 属于哪个网段；当存在多条可匹配路由时，通常遵循最长前缀匹配（掩码更长者优先）。
- **网关**：表示下一跳地址。由截图可见，本机存在两条默认路由：一条经网关 172.22.41.233 转发，另一条经网关 10.136.16.1 转发；而“在链路上”表示该目的网络为直连网络，无需经过下一跳。

- **接口**: 表示该路由项对应的本机出接口(出站 IP)。例如默认路由可能从 172.22.41.233 或 10.136.26.146 出口转发; 另外本机还存在虚拟网卡接口如 192.168.142.1、192.168.233.1 等。
- **跃点数**: 反映路径代价(可理解为优先级/开销)。当多条路由同时满足匹配时, 通常优先选择跃点数(Metric)较小的一条进行转发。

其次我们要分析的是 netstat 最常见的应用, 执行 netstat -a 来查看端口占用情况和监听状态。我们添加参数将地址和端口号数字化, 并指定所监听的协议为 TCP。执行 netstat -anp TCP 结果如下图所示。

```

C:\WINDOWS\system32\cmd.exe
C:\Users\19711>netstat -anp TCP

活动连接

协议 本地地址 外部地址 状态
TCP 0.0.0.0:135 0.0.0.0:0 LISTENING
TCP 0.0.0.0:445 0.0.0.0:0 LISTENING
TCP 0.0.0.0:902 0.0.0.0:0 LISTENING
TCP 0.0.0.0:912 0.0.0.0:0 LISTENING
TCP 0.0.0.0:3336 0.0.0.0:0 LISTENING
TCP 0.0.0.0:5040 0.0.0.0:0 LISTENING
TCP 0.0.0.0:7680 0.0.0.0:0 LISTENING
TCP 0.0.0.0:17164 0.0.0.0:0 LISTENING
TCP 0.0.0.0:22331 0.0.0.0:0 LISTENING
TCP 0.0.0.0:33212 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49664 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49665 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49668 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49669 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49672 0.0.0.0:0 LISTENING
TCP 0.0.0.0:49688 0.0.0.0:0 LISTENING
TCP 0.0.0.0:53737 0.0.0.0:0 LISTENING
TCP 0.0.0.0:64855 0.0.0.0:0 LISTENING
TCP 10.136.26.146:139 0.0.0.0:0 LISTENING
TCP 10.136.26.146:7680 10.136.17.114:53164 TIME_WAIT
TCP 10.136.26.146:50246 10.136.20.58:7680 TIME_WAIT
TCP 115.200.15.220:49862 122.248.50.155:7826 ESTABLISHED
TCP 115.200.15.220:49864 101.91.136.148:443 ESTABLISHED
TCP 115.200.15.220:50244 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:50245 10.136.2.154:7680 SYN_SENT
TCP 115.200.15.220:50996 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:50997 110.43.89.7:80 TIME_WAIT
TCP 115.200.15.220:51010 36.141.117.62:10101 TIME_WAIT
TCP 115.200.15.220:51011 36.141.117.62:10101 TIME_WAIT
TCP 115.200.15.220:51012 36.141.117.62:10101 TIME_WAIT
TCP 115.200.15.220:51013 36.141.117.62:10101 TIME_WAIT
TCP 115.200.15.220:52238 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:52315 120.92.48.232:80 TIME_WAIT
TCP 115.200.15.220:52329 183.131.147.48:443 ESTABLISHED
TCP 115.200.15.220:52339 113.240.75.249:443 TIME_WAIT
TCP 115.200.15.220:52672 180.111.196.239:443 ESTABLISHED
TCP 115.200.15.220:53603 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:54246 4.145.79.80:443 ESTABLISHED
TCP 115.200.15.220:54362 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:55296 180.153.202.34:80 ESTABLISHED
TCP 115.200.15.220:55297 60.188.68.55:80 CLOSE_WAIT
TCP 115.200.15.220:55379 61.151.229.189:443 CLOSE_WAIT
TCP 115.200.15.220:56571 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:56802 36.141.117.62:10101 TIME_WAIT
TCP 115.200.15.220:57602 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:57604 115.236.134.189:443 ESTABLISHED
TCP 115.200.15.220:57606 52.168.117.170:443 ESTABLISHED
TCP 115.200.15.220:57621 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:57623 36.141.117.62:10101 ESTABLISHED
TCP 115.200.15.220:57626 139.9.36.107:80 TIME_WAIT
TCP 115.200.15.220:57628 121.37.233.133:80 TIME_WAIT

```

图 8: netstat 查看 TCP 端口使用情况和监听状态执行结果

我们可以看到协议为 TCP 的监听端口和连接情况。我们对状态这一表项的各种取值举例分析：

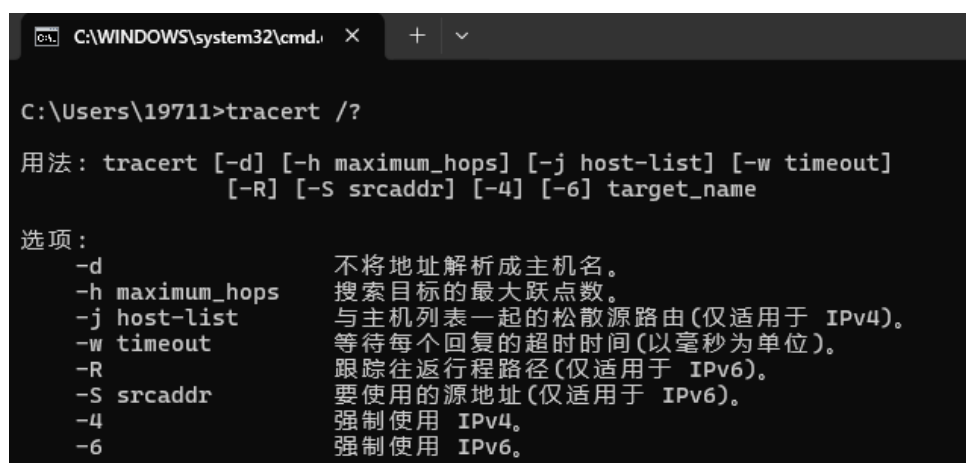
- **LISTENING**：表示本机正在监听某个 TCP 端口，等待远程主机发起连接。从截图可以看到，本机存在多个监听端口，例如 0.0.0.0:5040、0.0.0.0:445 等，其中 0.0.0.0 表示该端口在所有本地网络接口上监听。
- **ESTABLISHED**：表示 TCP 连接已经成功建立，通信双方可以正常进行数据传输。例如本机地址 115.200.15.220 的临时端口与远程主机 183.131.147.48:443、180.111.196.239:443 等建立了连接，根据端口号可以判断这些连接大多为 HTTPS 通信。
- **TIME_WAIT**：表示本机已主动关闭连接，并发送了最后一个 ACK 报文，当前连接处于 TIME_WAIT 状态。在该状态下，源 IP、源端口以及目的 IP、目的端口暂时不可复用，需要等待 **2MSL** 时间后，连接才会完全释放并转为 CLOSED 状态。
- **SYN_SENT**：表示本机已向远程主机发送 SYN 报文，请求建立 TCP 连接，但尚未收到对方的 SYN+ACK 响应，说明连接仍处于建立过程中。
- **CLOSE_WAIT**：表示远程主机已关闭连接，而本机尚未完成本地连接的关闭操作，通常需要等待应用程序关闭对应的套接字后，该状态才会消失。

2.1.5 tracert

功能、参数与原理

tracert 该命令通过发送带有不同 TLL 的 ICMP 数据包到目的地址来追踪经过的路由节点。TLL 的原理在 ping 命令中已经介绍过。

通过执行 tracert /?，可以查看该命令的参数。



```
C:\WINDOWS\system32\cmd.exe X + v

C:\Users\19711>tracert /?

用法: tracert [-d] [-h maximum_hops] [-j host-list] [-w timeout]
          [-R] [-S srcaddr] [-4] [-6] target_name

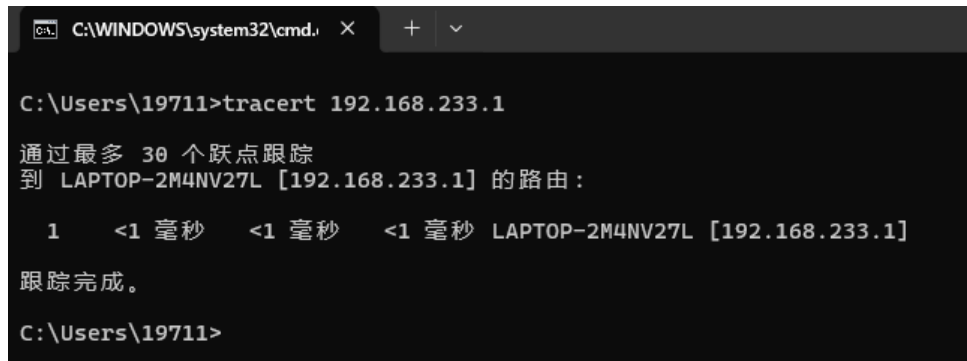
选项:
    -d          不将地址解析成主机名。
    -h maximum_hops  搜索目标的最大跃点数。
    -j host-list   与主机列表一起的松散源路由(仅适用于 IPv4)。
    -w timeout     等待每个回复的超时时间(以毫秒为单位)。
    -R           跟踪往返行程路径(仅适用于 IPv6)。
    -S srcaddr     要使用的源地址(仅适用于 IPv6)。
    -4           强制使用 IPv4。
    -6           强制使用 IPv6。
```

图 9: tracert /? 命令执行结果

在其基本功能的基础上，通过指定参数，我们可以指定最大路由跳数、超时时间等从而实现更加精准的网络诊断。

应用与分析

我们对同一个无线网络下的另一台主机进行 `tracert` 命令的测试。



```
C:\WINDOWS\system32\cmd.exe X + v

C:\Users\19711>tracert 192.168.233.1

通过最多 30 个跃点跟踪
到 LAPTOP-2M4NV27L [192.168.233.1] 的路由:

    1    <1 毫秒    <1 毫秒    <1 毫秒  LAPTOP-2M4NV27L [192.168.233.1]

跟踪完成。

C:\Users\19711>
```

图 10: `tracert` 命令执行结果

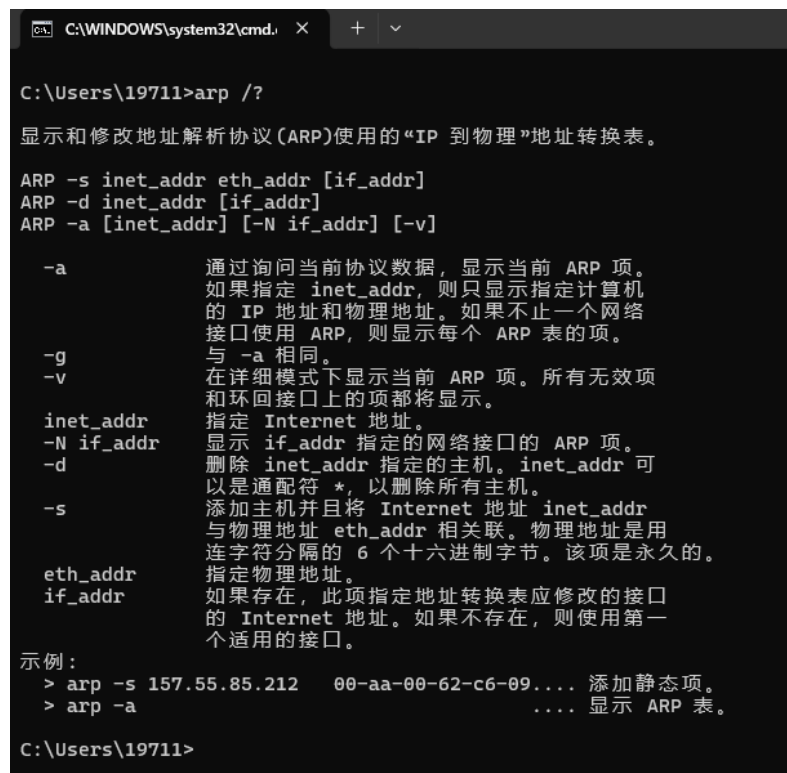
我们可以看到由于两台电脑属于同一个路由下的局域网，期间没有经过其它路由器转发。

2.1.6 arp

功能、参数与原理

ARP 协议通过解析 IP 地址获取其对应的 MAC 地址。`arp` 命令可以让我们查看到本机的 ARP 缓存内容，并且允许我们对 ARP 缓存进行修改。

通过执行 `arp /?`，可以查看该命令的参数。



```
C:\WINDOWS\system32\cmd.exe X + v

C:\Users\19711>arp /?

显示和修改地址解析协议(ARP)使用的“IP 到物理”地址转换表。

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

-a          通过询问当前协议数据，显示当前 ARP 项。
             如果指定 inet_addr，则只显示指定计算机
             的 IP 地址和物理地址。如果不止一个网络
             接口使用 ARP，则显示每个 ARP 表的项。
-g          与 -a 相同。
-v          在详细模式下显示当前 ARP 项。所有无效项
             和环回接口上的项都将显示。
inet_addr   指定 Internet 地址。
-N if_addr  显示 if_addr 指定的网络接口的 ARP 项。
-d          删除 inet_addr 指定的主机。inet_addr 可
             以是通配符 *，以删除所有主机。
-s          添加主机并且将 Internet 地址 inet_addr
             与物理地址 eth_addr 相关联。物理地址是用
             连字符分隔的 6 个十六进制字节。该项是永久的。
eth_addr    指定物理地址。
if_addr     如果存在，此项指定地址转换表应修改的接口
             的 Internet 地址。如果不存在，则使用第一
             个适用的接口。

示例：
> arp -s 157.55.85.212 00-aa-00-62-c6-09.... 添加静态项。
> arp -a          .... 显示 ARP 表。

C:\Users\19711>
```

图 11: `arp /?` 命令执行结果

应用与分析

我们执行 `arp -a`，查看本机的 ARP 缓存内容，结果如下图所示。

```

C:\WINDOWS\system32\cmd.exe
C:\Users\19711>arp -a

接口: 192.168.233.1 --- 0x7
Internet 地址      物理地址      类型
192.168.233.254    00-50-56-f4-75-fe    动态
192.168.233.255    ff-ff-ff-ff-ff-ff    静态
224.0.0.0          01-00-5e-00-00-00    静态
224.0.0.22         01-00-5e-00-00-16    静态
224.0.0.251        01-00-5e-00-00-fb    静态
224.0.0.252        01-00-5e-00-00-fc    静态
229.255.255.250    01-00-5e-7f-ff-fa    静态
230.0.0.1          01-00-5e-00-00-01    静态
239.255.255.250    01-00-5e-7f-ff-fa    静态
255.255.255.255    ff-ff-ff-ff-ff-ff    静态

接口: 192.168.142.1 --- 0xb
Internet 地址      物理地址      类型
192.168.142.254    00-50-56-f7-8c-e0    动态
192.168.142.255    ff-ff-ff-ff-ff-ff    静态
224.0.0.0          01-00-5e-00-00-00    静态
224.0.0.22         01-00-5e-00-00-16    静态
224.0.0.251        01-00-5e-00-00-fb    静态
224.0.0.252        01-00-5e-00-00-fc    静态
229.255.255.250    01-00-5e-7f-ff-fa    静态
230.0.0.1          01-00-5e-00-00-01    静态
239.255.255.250    01-00-5e-7f-ff-fa    静态
255.255.255.255    ff-ff-ff-ff-ff-ff    静态

接口: 10.136.26.146 --- 0x11
Internet 地址      物理地址      类型
10.136.16.1        2c-9d-1e-0d-b3-55    动态
10.136.16.24       98-5f-41-e9-b5-1f    动态
10.136.16.65       98-5f-41-aa-65-36    动态
10.136.16.147      c8-5e-a9-29-a9-05    动态
10.136.17.64       b2-2a-1a-c8-50-9a    动态
10.136.18.158      7c-50-79-80-ba-a1    动态
10.136.19.230      14-5a-fc-1b-1a-eb    动态
10.136.20.226      5c-b4-7e-a0-49-61    动态
10.136.22.46       2c-9d-1e-0d-b3-55    动态
10.136.23.137      e8-fb-1c-f7-2e-7b    动态
10.136.23.203      98-af-65-c9-30-d3    动态
10.136.24.37       4a-f9-4b-62-f8-c0    动态
10.136.24.202      c4-3d-1a-92-4a-0a    动态
10.136.25.184      30-89-4a-dc-3c-37    动态
10.136.28.46       6c-f6-da-17-61-5a    动态
10.136.29.234      14-75-5b-77-2a-fe    动态
10.136.30.169      c4-d0-e3-6b-ee-19    动态
10.136.31.255      ff-ff-ff-ff-ff-ff    静态
224.0.0.22         01-00-5e-00-00-16    静态
224.0.0.251        01-00-5e-00-00-fb    静态
224.0.0.252        01-00-5e-00-00-fc    静态
255.255.255.255    ff-ff-ff-ff-ff-ff    静态

接口: 172.22.41.233 --- 0x16
Internet 地址      物理地址      类型
172.22.0.1         58-60-5f-72-4c-91    动态
  
```

图 12: `arp` 命令查看当前 ARP 缓存执行结果片段

在上图中，有四个网络适配器，结合前面 `ipconfig` 的结果，可以知道第一个接口是无线网络的适配器。我们可以看到各个网络适配器中 IP 地址和 MAC 地址之间映射关系，类型表项中动态表示该映射会定期向网络中询问并进行更新，静态表示该映射不会自动进行更新。

2.1.7 telnet

功能、参数与原理

该命令可以使 telnet 客户端进入 telnet 服务端的终端，常用于远程登录和控制服务器。

由于 telnet 在 Win10 中默认是关闭的，我们需要在「控制面板」中的「打开或关闭 Windows 功能」中将其客户端打开。

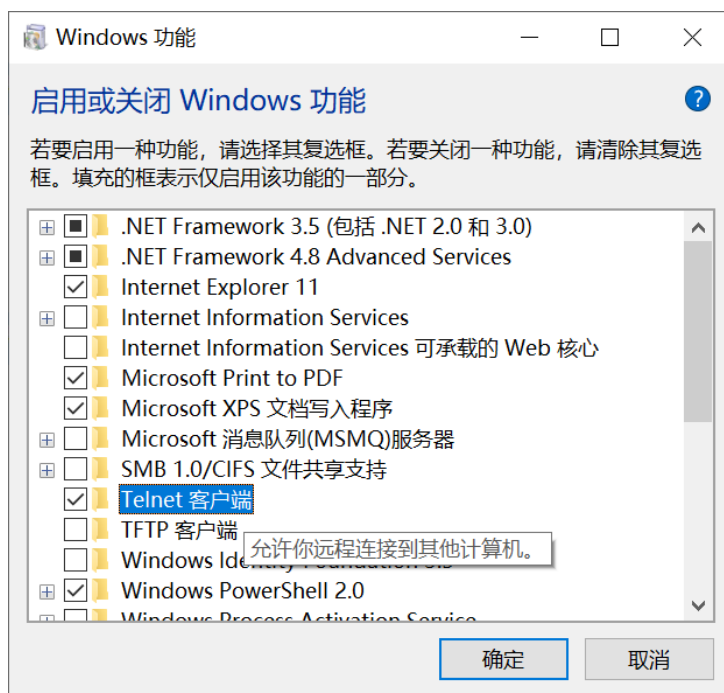


图 13: 打开 telnet 客户端功能

通过执行 telnet /?, 可以查看该命令的参数

```
C:\WINDOWS\system32\cmd.exe
C:\Users\19711>telnet /?

telnet [-a][-e escape char][-f log file][-l user][-t term][host [port]]
-a      企图自动登录。除了用当前已登陆的用户名以外，与 -l 选项相同。
-e      跳过字符来进入 telnet 客户端提示。
-f      客户端登录的文件名。
-l      指定远程系统上登录用的用户名。
-t      要求远程系统支持 TELNET ENVIRON 选项。
-t      指定终端类型。
        支持的终端类型是: vt100, vt52, ansi 和 vtnt。
host    指定要连接的远程计算机的主机名或 IP 地址。
port    指定端口号或服务名。
```

图 14: telnet /? 命令执行结果

可以看到，我们可以指定远程服务器的 ip 和端口进行登录，也可以指定参数以实现自动登录。在登录后的终端页面，输入指定的 Escape 字符（默认为 Ctrl +]），可以回退到 telnet 客户端界面。

进入 telnet 客户端页面，我们还可以查看针对管理页面的命令参数。

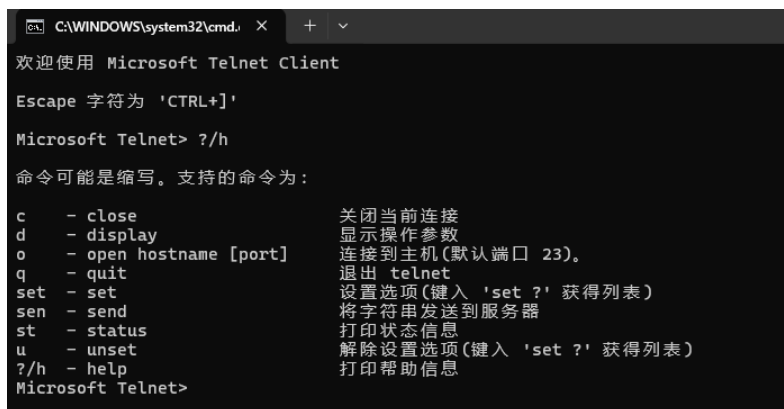


图 15: telnet 客户端帮助

通过不同的指令，我们可以建立、关闭远程连接，向远程服务器发送字符串等。

应用与分析

接下来，我们以虚拟机上的 ubuntu 服务器为例进行远程连接的测试。

通过查阅资料，先在 `ubuntu` 上开启了 `telnet` 服务端功能，并执行 `ifconfig` 来查看虚拟机的 `ip` 地址。在本机客户端执行 `telnet 192.168.35.128` 之后，输入账号密码，就进入了服务器的终端。

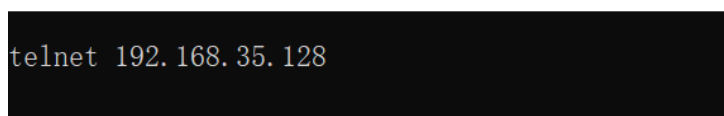


图 16: telnet 192.168.35.128

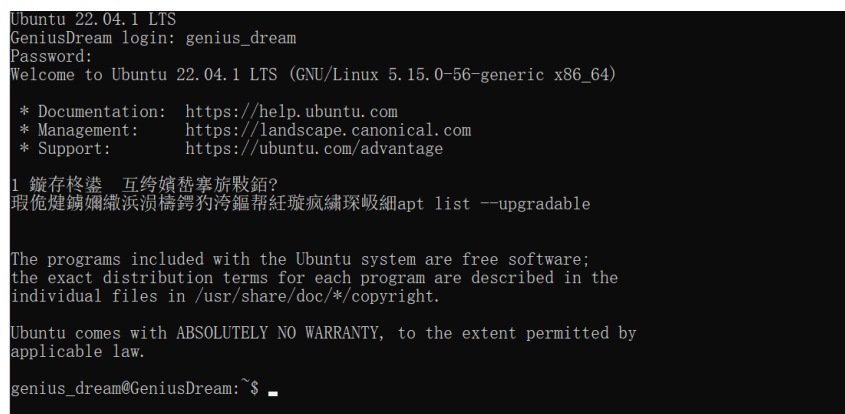


图 17: telnet 成功连接 ubuntu 服务器

默认的 **Escape** 键是 **Ctrl +]**，所以在上图的状态下按下输入 **Ctrl +]**，我们就可以回到客户端进行其他操作。

在客户端我们执行 `st` 查看当前的连接状态，执行 `send hello` 给服务器发送字符串“hello”，运行结果如下图。

```
Microsoft Telnet> st
已连接到 192.168.35.128
协商的终端类型为 ANSI
Microsoft Telnet> send hello
发送字符串 hello
```

图 18: telnet 客户端操作示例及运行结果

再次按下 Ctrl + V 和 Ctrl + M 可以回到远程终端环境。

2.2 交换机与路由器

2.2.1 课程要求

- (1) 安装 Packet Tracer，在 Packet Tracer 仿真环境下，熟悉交换机命令与交换机初始化配置。
- (2) 在交换机上实现 VLAN 配置；要求：创建三个 VLAN，给出拓扑，查看 VLAN 信息。
- (3) 基于 Console 控制台登录配置路由器，学习路由器配置相关命令。
- (4) 基于 Packet Tracer 构建网络环境，分别进行静态路由配置和基于 RIP 的动态路由配置；有余力的同学可设计基于 OSPF 的动态路由配置；能够进行综合集成网络情景设计更佳。

2.2.2 交换机命令学习及交换机初始化设置

交换机的基本作用

交换机是一种工作在数据链路层的网络设备，主要负责根据 MAC 地址表实现局域网内的数据转发。在本课程设计中，交换机主要用于构建局域网环境，为后续 VLAN 划分和跨 VLAN 通信实验提供基础平台。

交换机的必要性

为保证实验环境的可控性与可重复性，在进行具体配置前，需要对交换机进行初始化设置，清除可能存在的历史配置，统一设备状态，避免旧配置对实验结果产生干扰。

交换机的初始化设置

为完成后续 VLAN 划分及网络实验，需要首先对交换机进行初始化配置。以下列出了实验中常用的交换机基础命令及其功能说明。

(1) 模式切换类命令

这些命令用于在不同配置层级之间切换，是所有配置操作的前提。

- enable: 进入特权模式，用于查看和修改交换机配置。
- configure terminal: 进入全局配置模式，对交换机进行整体配置。

- `interface < 接口名 >`: 选择具体接口并进入端口配置模式，用于配置交换机端口属性。
- `exit`: 退出当前配置模式，返回上一级模式。
- `end`: 直接退出到特权模式，结束当前配置过程。

说明: 交换机配置具有层级结构，不同命令只能在对应模式下执行。

(2) 交换机初始化常用命令

该类命令主要用于完成交换机的基础初始化配置，是后续实验的前提。

- `hostname < 设备名 >`: 设置交换机名称，便于在多设备环境中区分不同交换机。
- `no ip domain-lookup`: 关闭 DNS 查询功能，避免命令输入错误时系统长时间等待解析。
- `show running-config`: 查看当前运行中的配置内容，用于检查初始化配置结果。
- `write` 或 `copy running-config startup-config`: 保存当前配置，防止设备重启后配置丢失。

(3) VLAN 创建与管理命令

该类命令用于创建和管理 VLAN，为后续 VLAN 实验提供基础环境。

- `vlan <VLAN_ID>`: 创建并进入指定 VLAN 的配置模式。
- `name <VLAN 名称 >`: 为 VLAN 设置名称，提高配置的可读性。
- `show vlan / show vlan brief`: 查看当前交换机上的 VLAN 信息及端口归属情况。

(4) 端口模式配置命令

该类命令用于配置交换机端口的工作模式及其 VLAN 属性。

- `switchport mode access`: 将端口设置为 Access 模式，用于连接终端设备。
- `switchport access vlan <VLAN_ID>`: 指定端口所属的 VLAN。
- `switchport mode trunk`: 将端口设置为 Trunk 模式，用于交换机之间或交换机与路由器之间传递多个 VLAN 数据。

通过上述初始化命令，对交换机的基本运行环境进行了统一设置，确保设备处于干净、稳定的状态，为后续 VLAN 划分及路由实验提供可靠的实验基础。

本部分通过对交换机常用命令的学习与初始化配置，掌握了交换机的基本操作流程和配置方法，为后续网络功能实验奠定了基础。

2.2.3 交换机及 VLAN 配置

网络拓扑结构与划分方案

根据课程要求，我们在单交换机基础上构建三个 VLAN 进行试验，为了进一步进行交换机的知识学习，在原有课程要求的基础上，在跨交换机基础上也构建三个 VLAN 进行试验。

表 1: 各主机 IP 地址的划分

主机名称	IP 地址	子网掩码	网关
FPH51-PC0	192.168.10.2	255.255.255.0	缺省
FPH51-PC1	192.168.20.2	255.255.255.0	缺省
FPH51-PC2	192.168.30.2	255.255.255.0	缺省
FPH51-PC3	192.168.10.3	255.255.255.0	缺省
FPH51-PC4	192.168.20.3	255.255.255.0	缺省
FPH51-PC5	192.168.30.3	255.255.255.0	缺省

(1) 单交换机

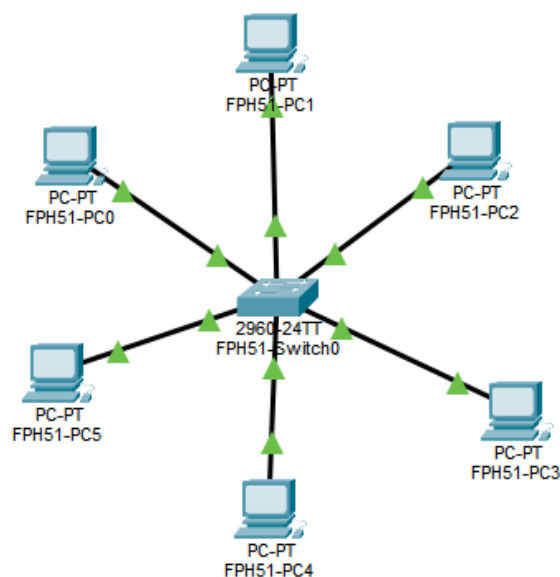


图 19: 单交换机网络拓扑图

(2) 跨交换机

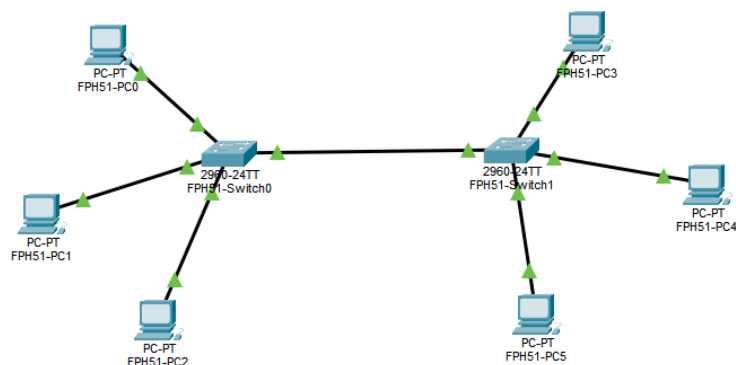


图 20: 跨交换机网络拓扑图

相关命令介绍

- `enable`: 进入特权模式。
- `configure terminal`: 进入全局配置模式。
- `interface`: 选择一个接口进行配置，进入端口配置模式。
- `exit`: 返回上一级。
- `end`: 直接返回到特权模式。
- `vlan`: 指定 VLAN ID，启用该 VLAN。
- `switchport access vlan`: 在端口配置模式下，设置该端口对应的 VLAN。
- `switchport mode trunk`: 在端口配置模式下，设置端口模式为 Trunk。
- `show vlan`: 查看 VLAN 配置。

配置过程与结果分析

由于交换机的配置都大小无异，我们这里只讲解单交换机情景下的配置过程，考虑到跨交换机配置雷同，为避免赘述，跨交换机情景只展示相关截图，不再重复文字赘述具体过程。

(1) 单交换机

单交换机配置过程如下，我们将 FPH51_0 和 FPH51_3 配置在 VLAN 10 下，FPH51_1 和 FPH51_4 配置在 VLAN 20 下，FPH51_2 和 FPH51_5 配置在 VLAN 30 下，具体命令操作如下：

<code>enable</code>	进入特权模式
<code>configure terminal</code>	进入全局配置模式
<code>vlan 10</code>	创建并启用 VLAN 10
<code>exit</code>	退出 VLAN 10 配置模式
<code>vlan 20</code>	创建并启用 VLAN 20
<code>exit</code>	退出 VLAN 20 配置模式
<code>vlan 30</code>	创建并启用 VLAN 30
<code>exit</code>	退出 VLAN 30 配置模式
<code>interface fastEthernet 0/1</code>	进入端口 0/1 的配置模式
<code>switchport mode access</code>	将端口设置为 Access 模式
<code>switchport access vlan 10</code>	将端口 0/1 划分到 VLAN 10
<code>exit</code>	退出端口 0/1 配置模式
<code>interface fastEthernet 0/2</code>	进入端口 0/2 的配置模式
<code>switchport mode access</code>	将端口设置为 Access 模式
<code>switchport access vlan 20</code>	将端口 0/2 划分到 VLAN 20
<code>exit</code>	退出端口 0/2 配置模式
<code>interface fastEthernet 0/3</code>	进入端口 0/3 的配置模式
<code>switchport mode access</code>	将端口设置为 Access 模式
<code>switchport access vlan 30</code>	将端口 0/3 划分到 VLAN 30
<code>exit</code>	退出端口 0/3 配置模式
<code>end</code>	结束交换机配置，返回特权模式
<code>show vlan</code>	查看 VLAN 配置并验证端口划分情况

为了简化操作，在 Packet Tracer 模拟环境下使用简化命令进行操作，详细简化命令如下表：

表 2: 常用交换机命令及其简写形式

命令全称	常用简写	命令全称	常用简写
<code>enable</code>	<code>en</code>	<code>configure terminal</code>	<code>conf t</code>
<code>interface</code>	<code>int</code>	<code>interface range</code>	<code>int ra</code>
<code>switchport mode access</code>	<code>sw mo acc</code>	<code>switchport access vlan</code>	<code>sw acc vlan</code>
<code>show vlan</code>	<code>sh vlan</code>	<code>show vlan brief</code>	<code>sh vlan br</code>
<code>exit</code>	<code>ex</code>		

```

Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#vlan 20
Switch(config-vlan)#vlan 30
Switch(config-vlan)#ex
Switch(config)#int ra fa0/1, fa0/4
Switch(config-if-range)#sw mo acc
Switch(config-if-range)#sw acc vlan 10
Switch(config-if-range)#ex
Switch(config)#int ra fa0/2, fa0/5
Switch(config-if-range)#sw mo acc
Switch(config-if-range)#sw acc vlan 20
Switch(config-if-range)#ex
Switch(config)#int ra fa0/3, fa0/6
Switch(config-if-range)#sw mo acc
Switch(config-if-range)#sw acc vlan 30
Switch(config-if-range)#ex
Switch(config)#

```

图 21: 单交换机 VLAN 配置过程

配置结束后，使用 show vlan 查看该交换机的配置情况，结果符合预期。

```

Switch(config)#ex
Switch#
%SYS-5-CONFIG_I: Configured from console by console
sh vlan

```

VLAN	Name	Status	Ports
1	default	active	Fa0/7, Fa0/8, Fa0/9, Fa0/10 Fa0/11, Fa0/12, Fa0/13, Fa0/14 Fa0/15, Fa0/16, Fa0/17, Fa0/18 Fa0/19, Fa0/20, Fa0/21, Fa0/22 Fa0/23, Fa0/24, Gig0/1, Gig0/2
10	VLAN0010	active	Fa0/1, Fa0/4
20	VLAN0020	active	Fa0/2, Fa0/5
30	VLAN0030	active	Fa0/3, Fa0/6
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

VLAN	Type	SAID	MTU	Parent	RingNo	BridgeNo	Stp	BrdgMode	Trans1	Trans2
1	enet	100001	1500	-	-	-	-	-	0	0
10	enet	100010	1500	-	-	-	-	-	0	0
20	enet	100020	1500	-	-	-	-	-	0	0
30	enet	100030	1500	-	-	-	-	-	0	0

--More--

图 22: 单交换机 VLAN 配置过程

(2) 跨交换机配置 Trunk 端口后，同一 VLAN 内的主机可实现跨交换机通信，跨交换机配置过程如下（两交换机一致，仅展示 FPH51_Switch0）:

```

Switch>en
Switch#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Switch(config)#vlan 10
Switch(config-vlan)#vlan 20
Switch(config-vlan)#vlan 30
Switch(config-vlan)#ex
Switch(config)#int fa0/1
Switch(config-if)#sw mo acc
Switch(config-if)#sw acc vlan 10
Switch(config-if)#ex
Switch(config)#int fa0/2
Switch(config-if)#sw mo acc
Switch(config-if)#sw acc vlan 20
Switch(config-if)#ex
Switch(config)#int fa0/3
Switch(config-if)#sw mo acc
Switch(config-if)#sw acc vlan 30
Switch(config-if)#ex
Switch(config)#int fa0/24
Switch(config-if)#sw mo tr
Switch(config-if)#ex
Switch(config)#

```

图 23: 跨交换机 Switch0 配置过程

配置结果如下图:

```
sh vlan br
```

VLAN	Name	Status	Ports
1	default	active	Fa0/4, Fa0/5, Fa0/6, Fa0/7 Fa0/8, Fa0/9, Fa0/10, Fa0/11 Fa0/12, Fa0/13, Fa0/14, Fa0/15 Fa0/16, Fa0/17, Fa0/18, Fa0/19 Fa0/20, Fa0/21, Fa0/22, Fa0/23 Gig0/1, Gig0/2
10	VLAN0010	active	Fa0/1
20	VLAN0020	active	Fa0/2
30	VLAN0030	active	Fa0/3
1002	fddi-default	active	
1003	token-ring-default	active	
1004	fddinet-default	active	
1005	trnet-default	active	

图 24: 跨交换机 VLAN 配置完成后检查配置文件

PC0 分别 ping 同一 VLAN 下的 PC3(192.168.10.3) 和不同 VLAN 下的 PC1 (192.168.20.2)，发现交换机两端同一 VLAN 的主机可以相互 ping 通，不同 VLAN 的主机 ping 不通，如下图：

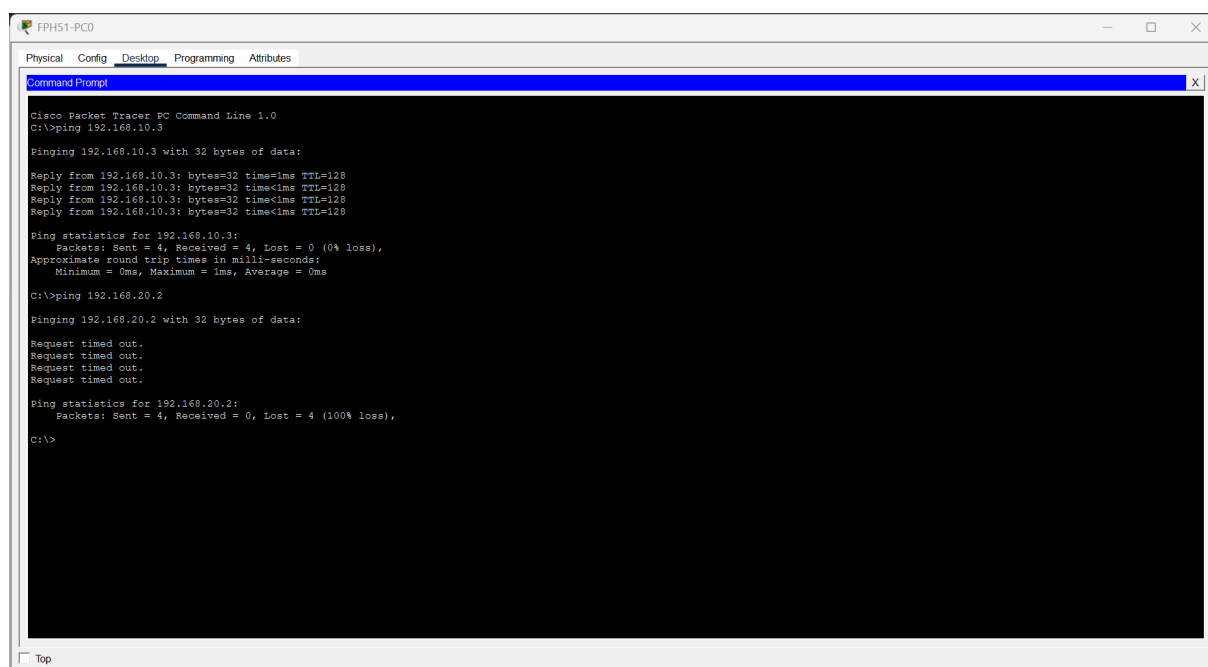


图 25: 主机 FPH51_PC0 ping 同一 VLAN 和不同 VLAN 下的主机

2.2.4 路由器命令学习及路由器初始化设置

路由器的基本作用

路由器是一种工作在网络层的网络设备，主要用于根据路由表实现不同网络之间的数据转发。在本课程设计中，路由器用于连接不同网段，实现跨网段通信以及静态路由和动态路由协议的配置，是构建复杂网络拓扑的重要组成部分。

路由器配置的必要性

在进行路由实验前，需要对路由器进行基础配置和初始化设置，以确保设备处于可控、稳定的工作状态。通过规范的初始化操作，可以避免历史配置对实验结果的干扰，并为后续路由协议配置提供统一的实验环境。

路由器的初始化设置

为完成后续静态路由及动态路由实验，需要首先对路由器进行初始化配置。以下列出了实验中常用的路由器基础命令及其功能说明。

(1) 模式切换类命令

该类命令用于在不同配置层级之间切换，是路由器配置操作的基础。

- `enable`: 进入特权模式，用于查看路由器状态及进行高级配置。
- `configure terminal`: 进入全局配置模式，对路由器进行整体参数设置。
- `interface < 接口名 >`: 进入指定接口的配置模式，用于配置接口的 IP 地址及状态。
- `exit`: 退出当前配置模式，返回上一级模式。
- `end`: 直接返回特权模式，结束当前配置过程。

说明：路由器配置同样采用分层结构，不同类型的命令只能在对应的配置模式下执行。

(2) 路由器初始化常用命令

该类命令主要用于完成路由器的基础初始化设置，为后续路由功能配置提供前提条件。

- `hostname < 设备名 >`: 设置路由器名称，便于在多路由器环境中区分不同设备。
- `no ip domain-lookup`: 关闭 DNS 查询功能，避免因误输入命令导致系统长时间等待解析。
- `show running-config`: 查看当前运行配置，用于检查初始化配置是否正确。
- `write` 或 `copy running-config startup-config`: 保存当前配置，防止设备重启后配置丢失。

(3) 接口配置相关命令

该类命令用于配置路由器接口的 IP 参数及接口状态，是实现网络互联的基础。

- `interface g0/0 / interface f0/0`: 进入指定物理接口的配置模式。
- `ip add <IP 地址> <子网掩码>`: 为接口配置 IP 地址及子网掩码。
- `no shutdown`: 启用接口，使接口由关闭状态变为工作状态。

(4) 路由查看与验证命令

该类命令用于查看路由器当前的接口状态和路由信息，是验证配置结果的重要手段。

- `show ip interface brief`: 查看各接口的 IP 地址及接口状态。
- `show ip route`: 查看路由器的路由表信息，用于验证路由配置是否生效。
- `show running-config`: 检查当前路由器的整体配置情况。

通过上述路由器初始化配置命令，对路由器的基本运行环境和接口参数进行了统一设置，为后续静态路由和动态路由协议的配置提供了可靠的基础条件。

本部分通过对路由器常用命令及登录配置方法的学习，掌握了路由器的基本操作流程和初始化配置方法，为后续路由功能实验的顺利开展奠定了基础。

2.2.5 静态路由配置

网络拓扑结构与划分方案

根据课程要求，我们进行静态路由的配置。

网络拓扑结构如下图：

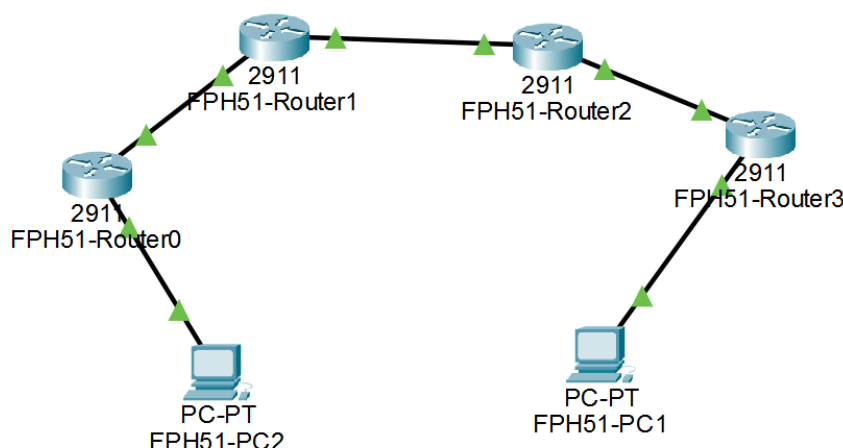


图 26: 静态路由配置拓扑图

网络划分方案如下：

表 3: 静态路由实验网络地址规划表

设备名称	接口名称	IP 地址	子网掩码	默认网关
FPH51_PC1	FastEthernet0	192.168.4.2	255.255.255.0	192.168.4.1
FPH51_PC2	FastEthernet0	192.168.1.2	255.255.255.0	192.168.1.1
FPH51_Router0	GigabitEthernet0/0	192.168.1.1	255.255.255.0	—
FPH51_Router0	GigabitEthernet0/1	10.0.12.1	255.255.255.0	—
FPH51_Router1	GigabitEthernet0/0	10.0.12.2	255.255.255.0	—
FPH51_Router1	GigabitEthernet0/1	10.0.23.1	255.255.255.0	—
FPH51_Router2	GigabitEthernet0/0	10.0.23.2	255.255.255.0	—
FPH51_Router2	GigabitEthernet0/1	10.0.34.1	255.255.255.0	—
FPH51_Router3	GigabitEthernet0/0	10.0.34.2	255.255.255.0	—
FPH51_Router3	GigabitEthernet0/1	192.168.4.1	255.255.255.0	—

相关命令介绍

- `enable`: 进入特权模式。
- `configure terminal`: 进入全局配置模式。
- `interface`: 选择一个接口进行配置, 进入端口配置模式。
- `line`: 指定参数后进入行配置模式。
- `exit`: 返回上一级模式。
- `end`: 直接返回到特权模式。
- `show ip route`: 查看路由器的路由表项。
- `ip route add NETMASK NEXTHOP`: 更新路由表项。

配置过程与结果分析

通过依次配置这四个路由, 我们将使得 FPH51_PC1 与 FPH51_PC2 能够相互通信。

配置路由 **FPH51_Router0**:

<code>enable</code>	进入特权模式
<code>configure terminal</code>	进入全局配置模式
<code>ip route 192.168.4.0 255.255.255.0 10.0.23.1</code>	配置到 192.168.4.0/24 网络的静态路由
<code>exit</code>	退出全局配置模式
<code>show ip route</code>	查看并验证静态路由配置结果

配置路由 **FPH51_Router1**:

<code>enable</code>	进入特权模式
<code>configure terminal</code>	进入全局配置模式
<code>ip route 192.168.1.0 255.255.255.0 10.0.23.2</code>	配置到 192.168.1.0/24 网络的静态路由
<code>ip route 192.168.4.0 255.255.255.0 10.0.23.2</code>	配置到 192.168.4.0/24 网络的静态路由
<code>exit</code>	退出全局配置模式
<code>show ip route</code>	查看并验证静态路由配置结果

配置路由 **FPH51_Router2**:

<code>enable</code>	进入特权模式
<code>configure terminal</code>	进入全局配置模式
<code>ip route 192.168.4.0 255.255.255.0 10.0.23.1</code>	配置到 192.168.4.0/24 网络的静态路由
<code>exit</code>	退出全局配置模式
<code>show ip route</code>	查看并验证静态路由配置结果

配置路由 **FPH51_Router3**:

enable	进入特权模式
configure terminal	进入全局配置模式
ip route 192.168.1.0 255.255.255.0 192.168.4.2	配置到 192.168.1.0/24 网络的静态路由
exit	退出全局配置模式
show ip route	查看并验证静态路由配置结果

配置完成后，我们可以使用 show ip route 查看各路由器的路由表以验证是否配置成功。以 FPH51_Router0 为例，其路由表如下：

```

Router(config)#ex
Router#
%SYS-5-CONFIG_I: Configured from console by console
show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

    10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
      C    10.0.12.0/24 is directly connected, GigabitEthernet0/1
      L    10.0.12.1/32 is directly connected, GigabitEthernet0/1
      C    192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
        C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
        L    192.168.1.1/32 is directly connected, GigabitEthernet0/0
        S    192.168.4.0/24 [1/0] via 10.0.12.2

```

图 27: FPH51_Router0 的路由表

静态路由配置成功后，我们使用 ping 命令来验证 FPH51_PC1 和 FPH51_PC2 的可达性，结果符合我们的预期。

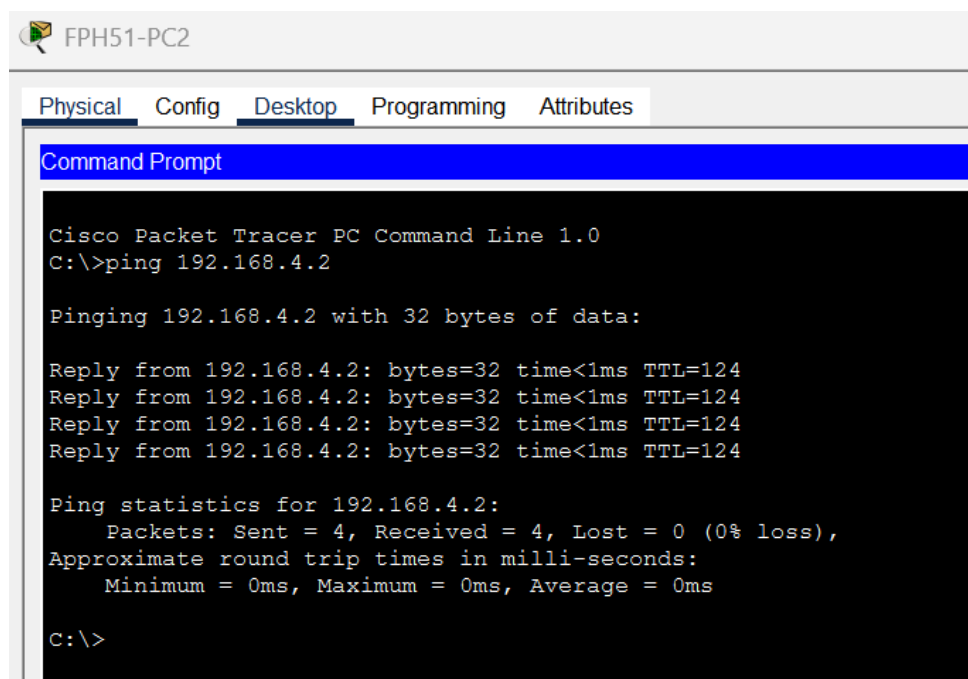


图 28: 静态路由配置结果验证

2.2.6 动态路由配置 RIP

网络拓扑结构与划分方案

根据课程要求，我们使用 RIP 配置动态路由。

如图所示，我们构造了以下网络拓扑结构：

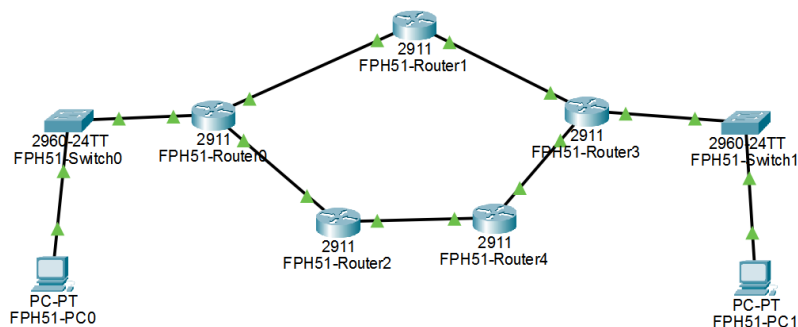


图 29: RIP 动态路由配置拓扑结构图

表 4: 静态路由实验网络地址规划表

设备名称	接口名称	IP 地址	子网掩码	默认网关
FPH51_PC0	FastEthernet0	192.168.1.2	255.255.255.0	192.168.1.1
FPH51_PC1	FastEthernet0	192.168.4.2	255.255.255.0	192.168.4.1
FPH51_Router0	GigabitEthernet0/0	192.168.1.1	255.255.255.0	—
FPH51_Router0	GigabitEthernet0/1	10.0.12.1	255.255.255.0	—
FPH51_Router0	GigabitEthernet0/2	10.0.13.1	255.255.255.0	—
FPH51_Router1	GigabitEthernet0/0	10.0.12.2	255.255.255.0	—
FPH51_Router1	GigabitEthernet0/1	10.0.24.1	255.255.255.0	—
FPH51_Router2	GigabitEthernet0/0	10.0.13.2	255.255.255.0	—
FPH51_Router2	GigabitEthernet0/1	10.0.35.1	255.255.255.0	—
FPH51_Router3	GigabitEthernet0/0	10.0.24.2	255.255.255.0	—
FPH51_Router3	GigabitEthernet0/1	10.0.54.2	255.255.255.0	—
FPH51_Router3	GigabitEthernet0/2	192.168.4.1	255.255.255.0	—
FPH51_Router4	GigabitEthernet0/0	10.0.35.2	255.255.255.0	—
FPH51_Router4	GigabitEthernet0/1	10.0.54.1	255.255.255.0	—

相关命令介绍

- `show ip route`: 查看路由器的路由表项。
- `router rip`: 进入 RIP 动态路由配置模式。
- `net IP`: 在 RIP 配置模式下设置邻接设备地址。
- `version VERSION`: 在 RIP 配置模式下设置 RIP 版本。
- `ip routing`: 开启路由功能。

配置过程与结果分析

通过依次配置这五个路由为RIP动态路由后,我们将使得FPH51_PC0与FPH51_PC1能够相互通信。不同于静态路由的是,他们之间进行通信时,传输的IP报将选择最少跳的路线到达目的地址。

配置路由 **FPH51_Router0:**

enable	进入特权模式
configure terminal	进入全局配置模式
interface g0/0	进入接口 g0/0 配置模式
ip add 192.168.1.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
exit	退出接口配置模式
interface g0/1	进入接口 g0/1 配置模式
ip add 10.0.12.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
exit	退出接口配置模式
interface g0/2	进入接口 g0/2 配置模式
ip add 10.0.13.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
end	返回特权模式
configure terminal	进入全局配置模式
router rip	进入 RIP 配置模式
version 2	设置 RIP 版本为 v2
no auto-summary	关闭自动汇总
net 192.168.1.0	宣告直连网络
net 10.0.12.0	宣告直连网络
net 10.0.13.0	宣告直连网络
end	返回特权模式

配置路由 **FPH51_Router1:**

enable	进入特权模式
configure terminal	进入全局配置模式
interface g0/0	进入接口 g0/0 配置模式
ip add 10.0.12.2 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
exit	退出接口配置模式
interface g0/1	进入接口 g0/1 配置模式
ip add 10.0.24.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
end	返回特权模式

configure terminal	进入全局配置模式
router rip	进入 RIP 配置模式
version 2	设置 RIP 版本为 v2
no auto-summary	关闭自动汇总
net 10.0.12.0	宣告直连网络
net 10.0.24.0	宣告直连网络
end	返回特权模式

配置路由 FPH51_Router2:

enable	进入特权模式
configure terminal	进入全局配置模式
interface g0/0	进入接口 g0/0 配置模式
ip add 10.0.13.2 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
exit	退出接口配置模式
interface g0/1	进入接口 g0/1 配置模式
ip add 10.0.35.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
end	返回特权模式
configure terminal	进入全局配置模式
router rip	进入 RIP 配置模式
version 2	设置 RIP 版本为 v2
no auto-summary	关闭自动汇总
net 10.0.13.0	宣告直连网络
net 10.0.35.0	宣告直连网络
end	返回特权模式

配置路由 FPH51_Router3:

enable	进入特权模式
configure terminal	进入全局配置模式
interface g0/0	进入接口 g0/0 配置模式
ip add 10.0.35.2 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
exit	退出接口配置模式
interface g0/1	进入接口 g0/1 配置模式
ip add 10.0.54.1 255.255.255.0	配置接口 IP 地址
no shutdown	启用接口
end	返回特权模式

<code>configure terminal</code>	进入全局配置模式
<code>router rip</code>	进入 RIP 配置模式
<code>version 2</code>	设置 RIP 版本为 v2
<code>no auto-summary</code>	关闭自动汇总
<code>net 10.0.35.0</code>	宣告直连网络
<code>net 10.0.54.0</code>	宣告直连网络
<code>end</code>	返回特权模式

配置路由 **FPH51_Router4**:

<code>enable</code>	进入特权模式
<code>configure terminal</code>	进入全局配置模式
<code>interface g0/0</code>	进入接口 g0/0 配置模式
<code>ip add 10.0.24.2 255.255.255.0</code>	配置接口 IP 地址
<code>no shutdown</code>	启用接口
<code>exit</code>	退出接口配置模式
<code>interface g0/1</code>	进入接口 g0/1 配置模式
<code>ip add 10.0.54.2 255.255.255.0</code>	配置接口 IP 地址
<code>no shutdown</code>	启用接口
<code>exit</code>	退出接口配置模式
<code>interface g0/2</code>	进入接口 g0/2 配置模式
<code>ip add 192.168.4.1 255.255.255.0</code>	配置接口 IP 地址
<code>no shutdown</code>	启用接口
<code>end</code>	返回特权模式

<code>configure terminal</code>	进入全局配置模式
<code>router rip</code>	进入 RIP 配置模式
<code>version 2</code>	设置 RIP 版本为 v2
<code>no auto-summary</code>	关闭自动汇总
<code>net 10.0.24.0</code>	宣告直连网络
<code>net 10.0.54.0</code>	宣告直连网络
<code>net 192.168.4.0</code>	宣告直连网络
<code>end</code>	返回特权模式

然后将所有路由器都开启 RIP:

开启 **RIP** 的路由 **FPH51_Router0**:

```
configure terminal
router rip
version 2
no auto-summary
net 192.168.1.0
net 10.0.12.0
net 10.0.13.0
end
```

进入全局配置模式
进入 RIP 动态路由配置模式
设置 RIP 版本为 v2
关闭自动汇总功能
宣告直连网络 192.168.1.0/24
宣告直连网络 10.0.12.0/24
宣告直连网络 10.0.13.0/24
返回特权模式

开启 **RIP** 的路由 **FPH51_Router1:**

```
configure terminal
router rip
version 2
no auto-summary
net 10.0.12.0
net 10.0.24.0
end
```

进入全局配置模式
进入 RIP 动态路由配置模式
设置 RIP 版本为 v2
关闭自动汇总功能
宣告直连网络 10.0.12.0/24
宣告直连网络 10.0.24.0/24
返回特权模式

开启 **RIP** 的路由 **FPH51_Router2:**

```
configure terminal
router rip
version 2
no auto-summary
net 10.0.13.0
net 10.0.35.0
end
```

进入全局配置模式
进入 RIP 动态路由配置模式
设置 RIP 版本为 v2
关闭自动汇总功能
宣告直连网络 10.0.13.0/24
宣告直连网络 10.0.35.0/24
返回特权模式

开启 **RIP** 的路由 **FPH51_Router3:**

```
configure terminal
router rip
version 2
no auto-summary
net 10.0.35.0
net 10.0.54.0
end
```

进入全局配置模式
进入 RIP 动态路由配置模式
设置 RIP 版本为 v2
关闭自动汇总功能
宣告直连网络 10.0.35.0/24
宣告直连网络 10.0.54.0/24
返回特权模式

开启 **RIP** 的路由 **FPH51_Router4:**

configure terminal	进入全局配置模式
router rip	进入 RIP 动态路由配置模式
version 2	设置 RIP 版本为 v2
no auto-summary	关闭自动汇总功能
net 10.0.24.0	宣告直连网络 10.0.24.0/24
net 10.0.54.0	宣告直连网络 10.0.54.0/24
net 192.168.4.0	宣告直连网络 192.168.4.0/24
end	返回特权模式

当配置完所有设备后，我们可以使用 show ip route 来检验路由配置结果。以交换机 FPH51_Router0 为例，配置结果如下图所示：

```
Router>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

 10.0.0.0/8 is variably subnetted, 7 subnets, 2 masks
C       10.0.12.0/24 is directly connected, GigabitEthernet0/1
L       10.0.12.1/32 is directly connected, GigabitEthernet0/1
C       10.0.13.0/24 is directly connected, GigabitEthernet0/2
L       10.0.13.1/32 is directly connected, GigabitEthernet0/2
R       10.0.24.0/24 [120/1] via 10.0.12.2, 00:00:08, GigabitEthernet0/1
R       10.0.35.0/24 [120/1] via 10.0.13.2, 00:00:08, GigabitEthernet0/2
R       10.0.54.0/24 [120/2] via 10.0.12.2, 00:00:08, GigabitEthernet0/1
         [120/2] via 10.0.13.2, 00:00:08, GigabitEthernet0/2
 192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
```

图 30: RIP 路由配置后的路由表

我们可以看到，这个路由表是符合预期的。

```
Router>show ip route 192.168.4.0
Routing entry for 192.168.4.0/24
Known via "rip", distance 120, metric 2
  Redistributing via rip
  Last update from 10.0.12.2 on GigabitEthernet0/1, 00:00:16 ago
  Routing Descriptor Blocks:
    * 10.0.12.2, from 10.0.12.2, 00:00:16 ago, via GigabitEthernet0/1
      Route metric is 2, traffic share count is 1
```

图 31: RIP 路径选择

可以看到,FPH51_Router0 到 192.168.4.0 的 RIP 路由走的是 10.0.12.2,即 FPH51_Router1 这一条路径,因为这条路径跳数最短,为最短路,所以 RIP 动态路由下选择这条路


```

C:\>ping 192.168.4.2

Pinging 192.168.4.2 with 32 bytes of data:

Reply from 192.168.4.2: bytes=32 time=4ms TTL=125
Reply from 192.168.4.2: bytes=32 time<1ms TTL=125
Reply from 192.168.4.2: bytes=32 time<1ms TTL=125
Reply from 192.168.4.2: bytes=32 time<1ms TTL=125

Ping statistics for 192.168.4.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 4ms, Average = 1ms

```

图 32: RIP 动态路由验证截图

发现 FPH51_PC0 和 FPH51_PC1 可以 ping 的通, 说明成功实现了 RIP 动态路由并且两台 PC 联通。

2.2.7 动态路由配置 OSPF

OSPF 动态路由可以在前面已有的 RIP 动态路由基础上修改实现, 这里简略介绍, 不再进行赘述, 介绍基础操作过程:

在所有路由器上执行以下操作, 关闭 RIP 动态路由协议:

configure terminal	进入全局配置模式
no router rip	关闭 RIP 动态路由协议
end	返回特权模式

开启 **OSPF** 的路由 **FPH51_R0**:

configure terminal	进入全局配置模式
router ospf 1	创建并进入 OSPF 进程 1
net 192.168.1.0 0.0.0.255 area 0	宣告直连网络加入 Area 0
net 10.0.12.0 0.0.0.255 area 0	宣告直连网络加入 Area 0
net 10.0.13.0 0.0.0.255 area 0	宣告直连网络加入 Area 0
end	返回特权模式

开启 **OSPF** 的路由 **FPH51_R1**:

configure terminal	进入全局配置模式
router ospf 1	创建并进入 OSPF 进程 1
net 10.0.12.0 0.0.0.255 area 0	宣告直连网络加入 Area 0
net 10.0.24.0 0.0.0.255 area 0	宣告直连网络加入 Area 0
end	返回特权模式

开启 **OSPF** 的路由 **FPH51_R2**:

<code>configure terminal</code>	进入全局配置模式
<code>router ospf 1</code>	创建并进入 OSPF 进程 1
<code>net 10.0.13.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>net 10.0.35.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>end</code>	返回特权模式

开启 **OSPF** 的路由 **FPH51_R3**:

<code>configure terminal</code>	进入全局配置模式
<code>router ospf 1</code>	创建并进入 OSPF 进程 1
<code>net 10.0.35.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>net 10.0.54.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>end</code>	返回特权模式

开启 **OSPF** 的路由 **FPH51_R4**:

<code>configure terminal</code>	进入全局配置模式
<code>router ospf 1</code>	创建并进入 OSPF 进程 1
<code>net 10.0.24.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>net 10.0.54.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>net 192.168.4.0 0.0.0.255 area 0</code>	宣告直连网络加入 Area 0
<code>end</code>	返回特权模式

查看 FPH51_Router0 的路由表，发现表中含有 O，说明操作成功。

```
Router>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

10.0.0.0/8 is variably subnetted, 7 subnets, 2 masks
C    10.0.12.0/24 is directly connected, GigabitEthernet0/1
L    10.0.12.1/32 is directly connected, GigabitEthernet0/1
C    10.0.13.0/24 is directly connected, GigabitEthernet0/2
L    10.0.13.1/32 is directly connected, GigabitEthernet0/2
O    10.0.24.0/24 [110/2] via 10.0.12.2, 00:00:14, GigabitEthernet0/1
O    10.0.35.0/24 [110/2] via 10.0.13.2, 00:04:24, GigabitEthernet0/2
O    10.0.54.0/24 [110/3] via 10.0.12.2, 00:00:14, GigabitEthernet0/1
      [110/3] via 10.0.13.2, 00:00:14, GigabitEthernet0/2
192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C    192.168.1.0/24 is directly connected, GigabitEthernet0/0
L    192.168.1.1/32 is directly connected, GigabitEthernet0/0
```

图 33: OSPF 路由表

我们对 FPH51_Router0 进行如下命令：`int g0/1, ip ospf cost 100`

通过命令，我们对该路由器 cost 进行了设置，然后重新回到 FPH51_Router0 使用命令 `show ip route` 查看其路由，得到下属结果：

```
10.0.0.0/8 is variably subnetted, 7 subnets, 2 masks
C   10.0.12.0/24 is directly connected, GigabitEthernet0/1
L   10.0.12.1/32 is directly connected, GigabitEthernet0/1
C   10.0.13.0/24 is directly connected, GigabitEthernet0/2
L   10.0.13.1/32 is directly connected, GigabitEthernet0/2
O   10.0.24.0/24 [110/4] via 10.0.13.2, 00:00:01, GigabitEthernet0/2
O   10.0.35.0/24 [110/2] via 10.0.13.2, 00:05:50, GigabitEthernet0/2
O   10.0.54.0/24 [110/3] via 10.0.13.2, 00:00:01, GigabitEthernet0/2
192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C   192.168.1.0/24 is directly connected, GigabitEthernet0/0
L   192.168.1.1/32 is directly connected, GigabitEthernet0/0
O   192.168.4.0/24 [110/4] via 10.0.13.2, 00:00:01, GigabitEthernet0/2
```

图 34: OSPF 路由表

通过结果我们发现，修改 cost 后，OSPF 直接走 10.0.13.2，即改走长路径（R0 → R2 → R4 → R3）说明在相同拓扑条件下，配置 OSPF 动态路由协议。OSPF 能够基于接口 cost 自动计算最优路径，默认情况下选择代价较低的路径。当通过手动调整接口 cost 后，OSPF 能够重新计算路由并选择新的最优路径，体现了其较 RIP 更灵活、可控的选路机制。

2.2.8 综合网络情景设计

网络拓扑结构与划分方案

我们模拟学校年级办公室电脑的网络集成，功能设计如下：

- 每个年级划分一个 VLAN，每个 VLAN 下包含两台 PC。
- 同一楼层的电脑通过二层交换机连接，不同楼层的网络通过三层交换机连接，并配置 OSPF 动态路由。
- 二楼是一、二年级办公室，三楼是三、四年级办公室。
- 设置一台内部局域网 WEB 服务器，与学校内网路由器相连。
- 设置一台学校外部广域网 WEB 服务器，与学校外网路由器相连。
- 内网路由器与外网路由器相连，通过配置 NAT 转换，使学校电脑可以通过内网访问外网。

拓扑结构设计如下，包含 8 台 PC、2 台二层交换机、1 台三层交换机、2 台路由器、2 台服务器：

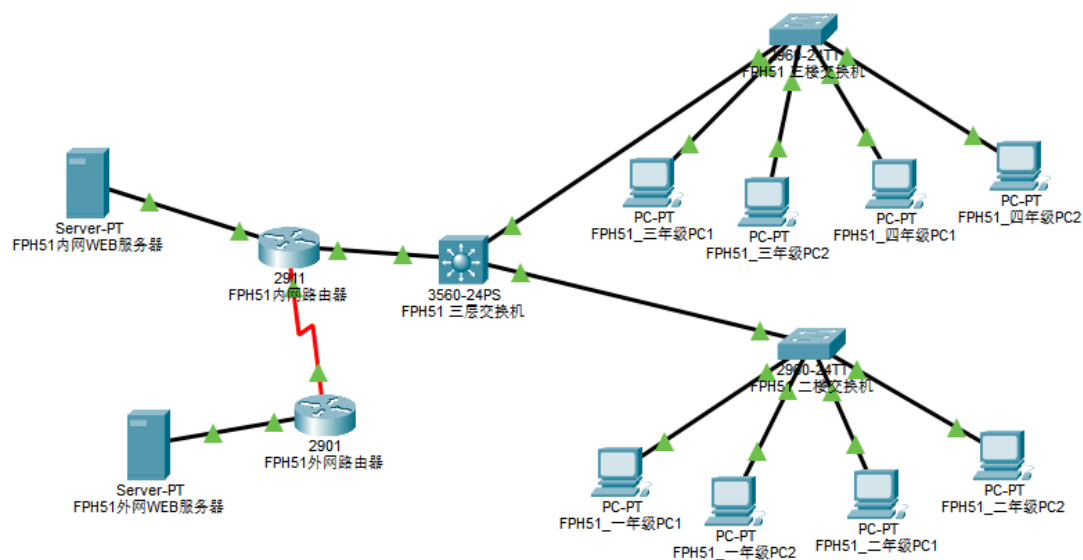


图 35: 综合网络情景设计拓扑结构

网络划分方案如下：

表 5: 综合网络设计实验网络地址规划表

设备名称	接口名称	IP 地址	子网掩码	默认网关
FPH51_一年级 PC1	FastEthernet0	192.168.1.1	255.255.255.0	192.168.1.254
FPH51_一年级 PC2	FastEthernet0	192.168.1.2	255.255.255.0	192.168.1.254
FPH51_二年级 PC1	FastEthernet0	192.168.2.1	255.255.255.0	192.168.2.254
FPH51_二年级 PC2	FastEthernet0	192.168.2.2	255.255.255.0	192.168.2.254
FPH51_三年级 PC1	FastEthernet0	192.168.3.1	255.255.255.0	192.168.3.254
FPH51_三年级 PC2	FastEthernet0	192.168.3.2	255.255.255.0	192.168.3.254
FPH51_四年级 PC1	FastEthernet0	192.168.4.1	255.255.255.0	192.168.4.254
FPH51_四年级 PC2	FastEthernet0	192.168.4.2	255.255.255.0	192.168.4.254
FPH51 二层交换机	VLAN1	192.168.1.254	255.255.255.0	—
FPH51 二层交换机	VLAN2	192.168.2.254	255.255.255.0	—
FPH51 二层交换机	VLAN3	192.168.3.254	255.255.255.0	—
FPH51 二层交换机	VLAN4	192.168.4.254	255.255.255.0	—
FPH51 三层交换机	FastEthernet0/3	192.168.10.2	255.255.255.0	—
FPH51 内网路由器	FastEthernet0/0	192.168.60.254	255.255.255.0	—
FPH51 内网路由器	FastEthernet1/0	192.168.10.1	255.255.255.0	—
FPH51 内网路由器	Serial2/0	201.1.8.1	255.255.255.0	—
FPH51 外网路由器	Serial2/0 (DCE)	201.1.8.10	255.255.255.0	—
FPH51 外网路由器	FastEthernet1/0	202.1.1.254	255.255.255.0	—
FPH51 内网 WEB 服务器	FastEthernet0	192.168.60.1	255.255.255.0	192.168.60.254
FPH51 外网 WEB 服务器	FastEthernet0	202.1.1.1	255.255.255.0	202.1.1.254

相关命令介绍

- access-list ID PERMISSION ADDR ANTIMASK: 指定访问控制列表;
- ip nat: 进行 NAT 相关配置;

配置过程与结果分析

设备 IP 地址的设置、交换机及路由器的基本配置我们在前面几个实验中已经详细讲解过了, 在这个实验中我们不再赘述。

本实验中两个特殊的配置如下: 在进行三层交换机的端口 ip 配置时, interface fastEthernet 默认进入的是端口的二层配置, 需要输入 no switchport 才能进入端口的三层进行路由相关的端口配置。

```
Switch(config)#int f 0/3
Switch(config-if)#no switchport
Switch(config-if)#ip address 192.168.10.2 255.255.255.0
```

图 36: 进入三层交换机的三层端口

L12-内网路由器的 NAT 配置:

enable	进入特权模式
configure terminal	进入全局配置模式
interface fastEthernet 1/0	进入子接口 1/0
ip nat outside	配置该接口为 NAT 外部接口
exit	返回上一级模式
interface serial 2/0	进入子接口 2/0
ip nat inside	配置该接口为 NAT 内部接口
exit	返回上一级模式
access-list 1 permit 192.168.10.0 0.0.0.255	对 LAN 构造一个访问控制列表
ip nat pool rebmit 201.1.8.2 201.1.8.2 netmask 255.0.0.0	配置可用的被映射 IP 地址池
ip nat inside source list 1 pool rebmit overload	建立映射并配置为 NAPT
end	返回特权模式

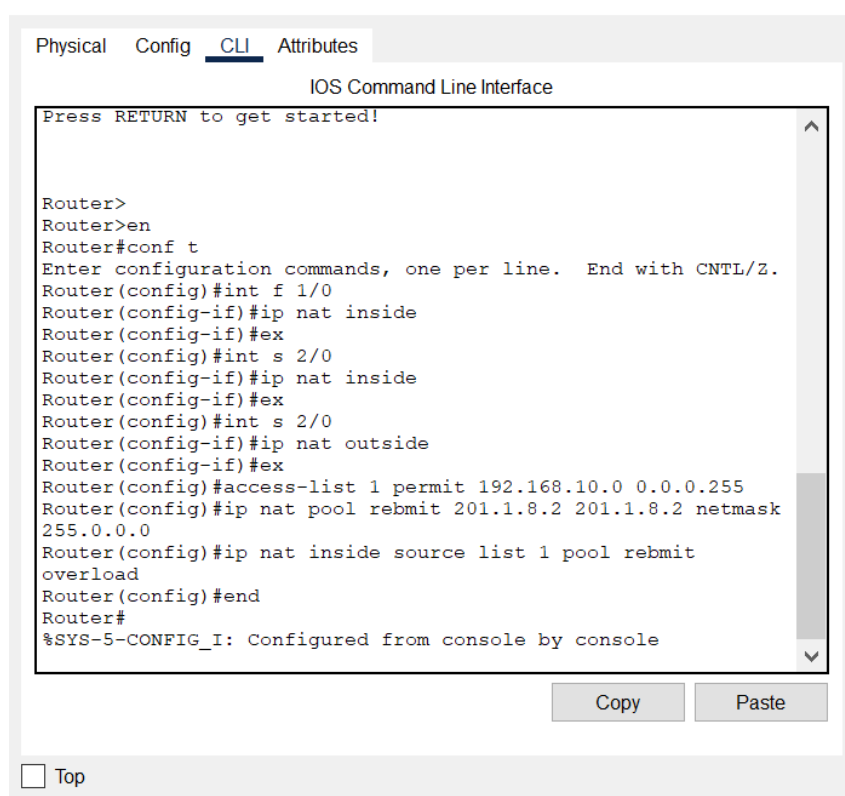


图 37: NAT 配置过程截图

所有设备都配置完成后，我们来测试一下年级办公室的电脑是否能上网了。
测试结果如下：

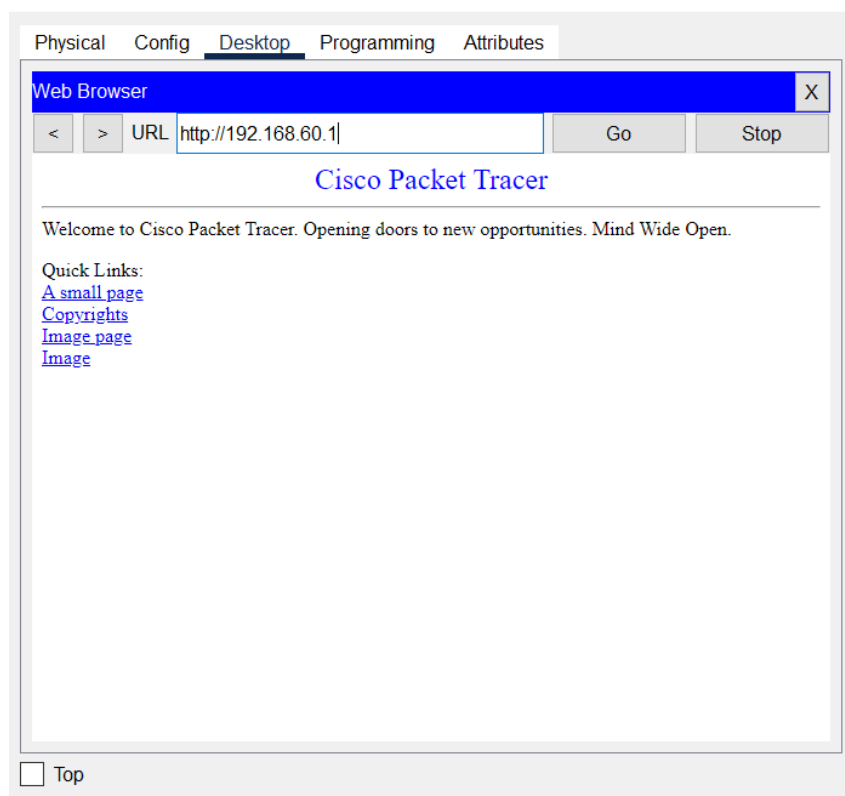


图 38: FPH51_一年级 PC1 通过 ip 访问内网

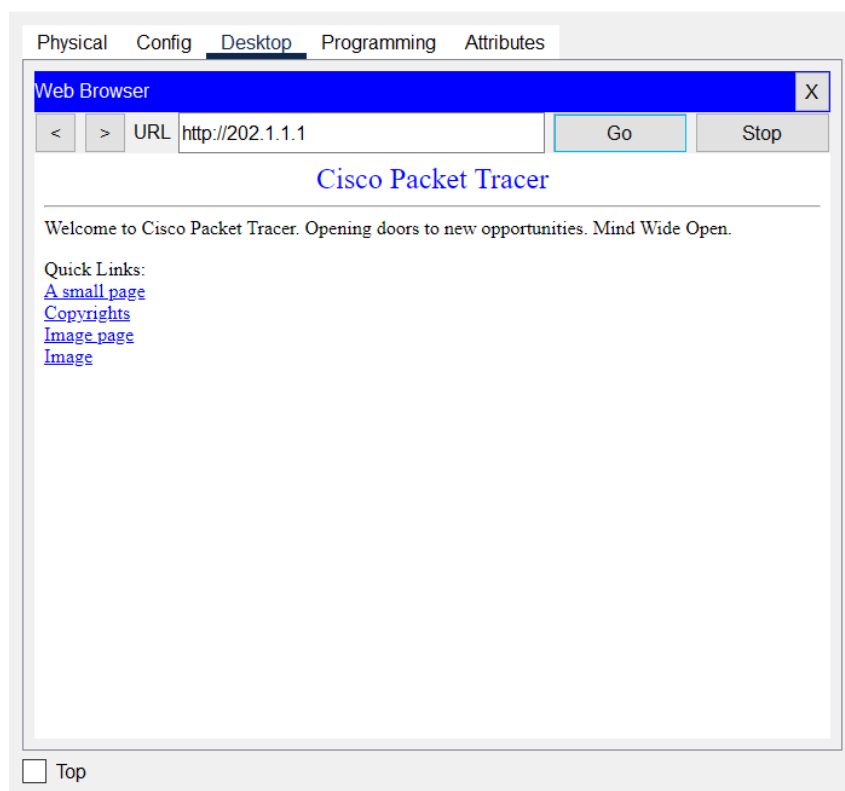


图 39: FPH51_一年级 PC1 通过 ip 访问外网

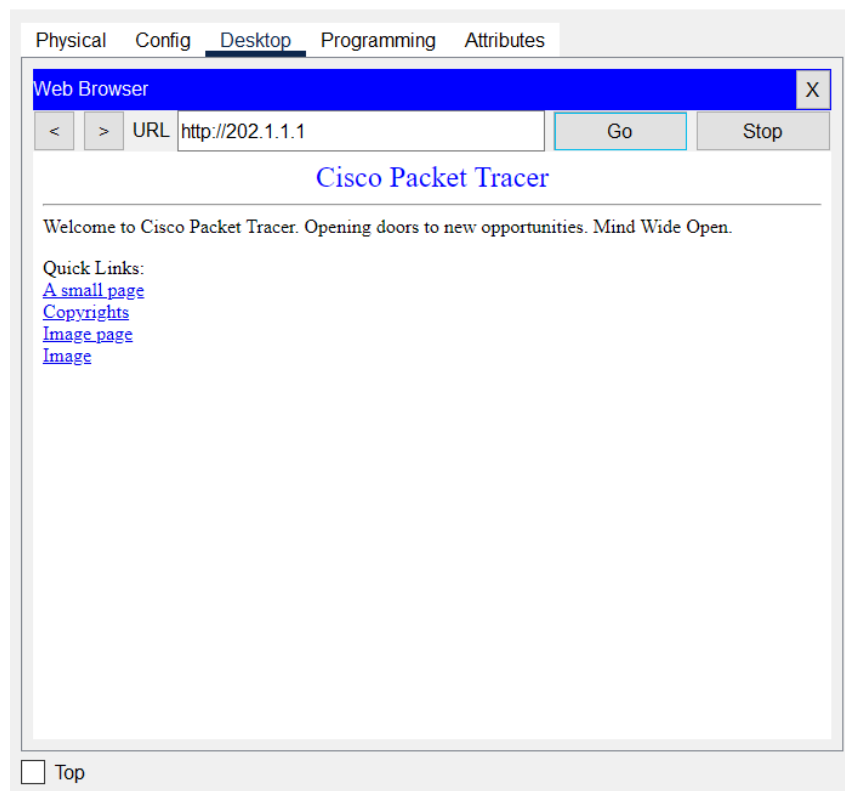


图 40: FPH51_一年级 PC1 通过域名访问外网

内网电脑能通过 ip 或者域名访问内网服务器、外网服务器，测试结果符合预期。

2.3 网络编程

2.3.1 课程要求

编程要求：捕获本机网卡的 IP 包，对捕获的 IP 包进行解析。要求必须输出以下字段：版本号、总长度、标识、标志位、片偏移、协议、源地址和目的地址。要有详细的说明文档，包括程序的设计思想、工作流程、关键问题、程序注释和对捕获包的解析截图。编程语言不作要求，可使用自己熟悉的 C、C++、java 或 C 等

2.3.2 设计思想

实验要求捕获本机网卡的 IP 数据包，并对其进行解析，编程语言不限。基于个人对编程语言的熟悉程度，我选择用 C++ 来完成本次网络编程的编写工作。

总体需要完成的工作有两项：第一项是捕获本机网卡 IP 包，第二项是解析 IP 包。

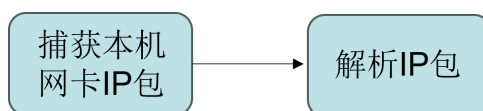


图 41: 网络编程总体设计思路

对于第一项工作，我们需要先获得本机网卡的 ip 地址，然后使用原始套接字（raw socket）绑定该 ip 地址来接收本机网卡上的数据包。

对于第二项工作，基于对 IP 数据包格式的了解，我们可以设计一个结构体指针来对 IP 数据包的头部进行逐字节的扫描解析。

2.3.3 实验环境

网络编程在 GCC 8.1.0 + MinGW-w64 8.1.0 环境下开发，使用 Visual Studio Code 开发套件，以下为开发及测试平台相关信息：

- 处理器：Intel(R) Core(TM) i7-14650HX (2.20 GHz)
- 内存：16.00GB
- Windows 11 家庭中文版，64 位操作系统, 基于 x64 的处理器

2.3.4 主要函数与结构体分析

抓包函数

这一部分使用 C++ 的 winSocks2 库实现，具体流程图如下：

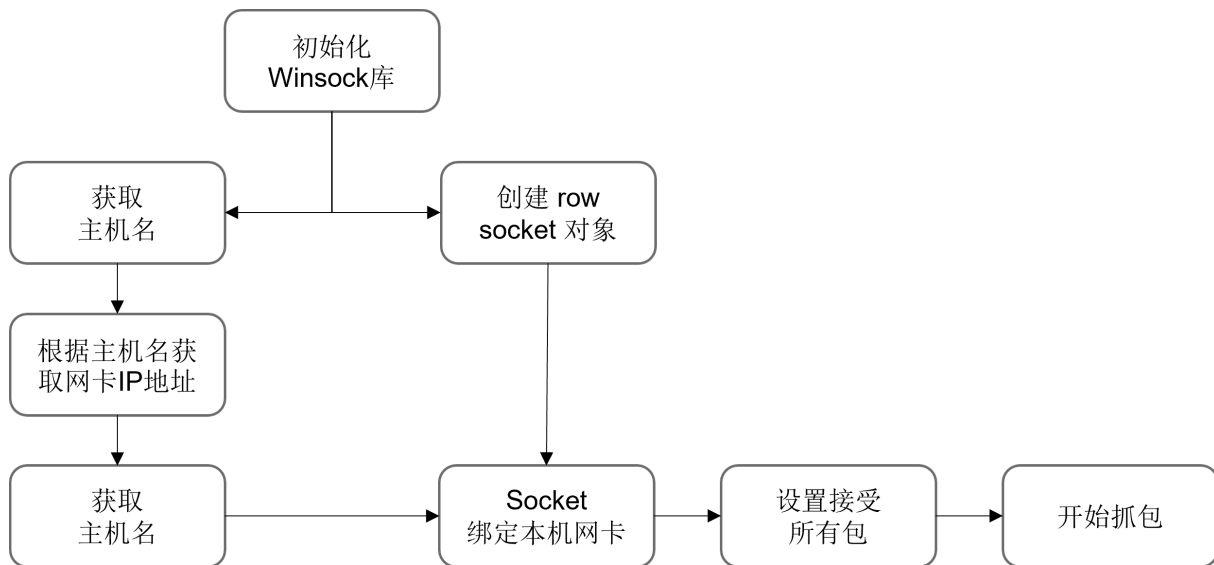


图 42: 抓包流程图

通过查阅文档，我们了解到了相应库函数的使用方式：

- 初始化 winsock 库：

```
1 WSAStartup(0x0202, &wsa_data);
```

- 获取主机名：

```
1 gethostname(hostname, maxHostNameLen);
```

- 根据主机名获取网卡 IP 地址：

```
1 HOSTENT *host = gethostbyname(hostname);
```

- 创建 SOCKADDR_IN 对象：

```
1 SOCKADDR_IN address{};
2 address.sin_addr.s_addr = *(unsigned long *)host->h_addr;
3 address.sin_family = AF_INET;
4 address.sin_port = htons(0);
```

- 创建 raw socket 对象：

```
1 SOCKET sockServer = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
```

- 绑定本机网卡:

```
1 bind(sockServer, (SOCKADDR *)&address, sizeof(SOCKADDR));
```

- 设置接收所有数据包:

```
1 WSAIoctl(sockServer, SIO_RCVALL,
2         &dwInBuff, sizeof(dwInBuff),
3         &dwOutBuff, sizeof(dwOutBuff),
4         &dwBytesRet, nullptr, nullptr);
```

- 开始抓包:

```
1 recv(sockServer, buffer, maxBufferSize, 0);
```

头部扫描结构体

该结构体用于解析 IP 数据报头部，不考虑选项字段，默认头部长度为 20 字节。
基于 IP 头部字段顺序，我们设计了如下结构体：

```
1 struct IP_datagram {
2     unsigned char Version_IHL;           // 版本 4 位 + 首部长 4 位
3     unsigned char TOS;                   // 服务类型
4     unsigned short TL;                   // 总长度
5     unsigned short id;                   // 标识
6     unsigned short flags_fragOffset;    // 标志位 3 位 + 片偏移 13 位
7     unsigned char TTL;                   // 生存时间
8     unsigned char protocol;              // 协议
9     unsigned short headerChecksum;       // 首部校验和
10    unsigned int srcAdd;                  // 源地址
11    unsigned int destAdd;                 // 目的地址
12};
```

为了便于输出字段的含义，我们设计了若干解析函数。

协议字段解析函数

```
1 std::string getProtocol() const {
2     switch (protocol) {
3         case 1: return "ICMP (1)";
4         case 2: return "IGMP (2)";
5         case 6: return "TCP (6)";
6         case 17: return "UDP (17)";
7         default: return std::to_string((int)protocol);
8     }
9 }
```

IP 版本解析函数

```
1 std::string getVersion() const {  
2     unsigned char version = Version_IHL >> 4;  
3     switch (version) {  
4         case 4: return "IPv4 (4)";  
5         case 6: return "IPv6 (6)";  
6         default: return std::to_string((int)version);  
7     }  
8 }
```

标志位解析函数

```
1 std::string getFlag() const {  
2     unsigned char flags = flags_fragOffset >> 13;  
3     std::string description;  
4     if (flags & 0b001) description += "more fragments, ";  
5     if (flags & 0b010) description += "don't fragment, ";  
6     if (description.empty()) return "0";  
7     description.pop_back();  
8     description.pop_back();  
9     description += " (" + std::to_string(flags) + ")";  
10    return description;  
11 }
```

2.3.5 程序流程图

程序完整流程图如下：

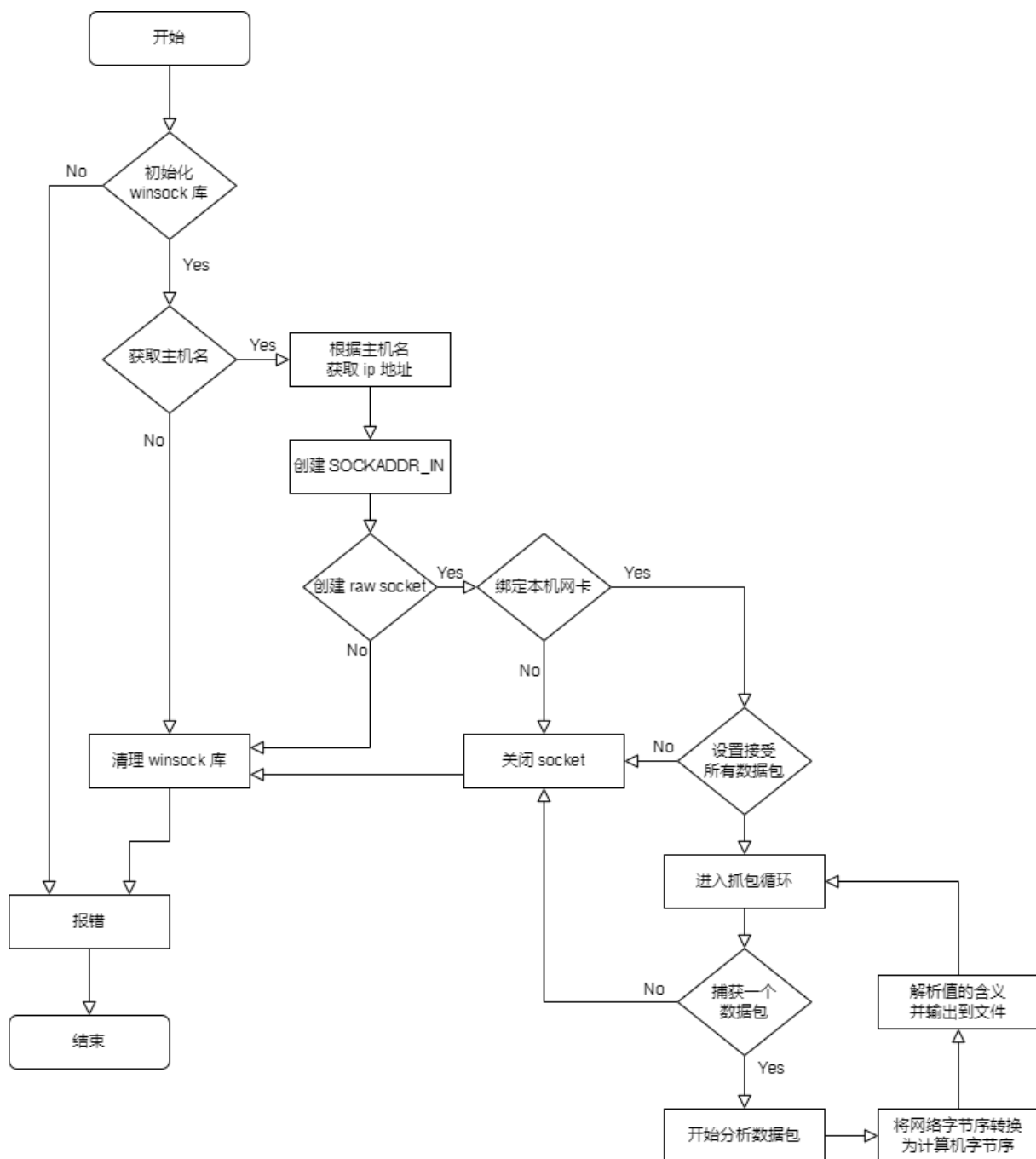


图 43: 程序完整流程图

2.3.6 难点解决

进行课程设计过程中遇到的难点主要有以下三个：

编译链接

在使用 winsock2 库头文件时，编译器报错找不到相应的库文件。

通过查阅资料，winsock2 作为动态链接库，在用 CMake 编译时，需要在 CMakeLists 中加入 `target_link_libraries(PacketCapture WS2_32)`，若使用 GCC 编译则需要在编译命令后加上参数 `-lWS2_32`。

管理员权限

刚开始进行抓包函数编写的时候，绑定网卡的函数 `bind()` 一直返回错误信息，经过查阅资料，得知 windows 下进行 socket 的绑定需要管理员权限。

用管理员权限打开 cmd 再进行编译操作即可解决。

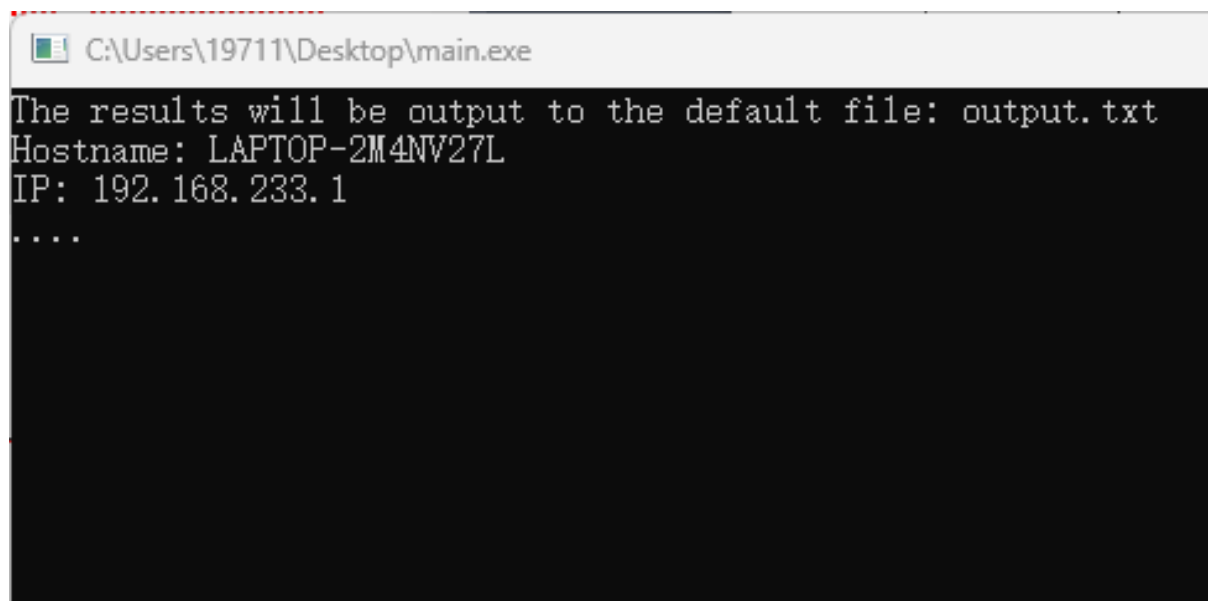
字节序问题

在写好 `IP_datagram` 结构体进行初步测试时，我们发现解析得到结果不太正常，具体表现有 IP 报的版本都不是 IPv4 和 IPv6，协议解析不出等。

通过查阅资料，我们了解到网络字节序与计算机字节序的差异，在大于 1 个字节的字段处理上，需要注意二者的转换。最终我们用 winsock2 库提供的 `htons`、`htonl`、`ntohs`、`ntohl` 用于字节序转换的函数解决了这个问题。

2.3.7 测试结果截图

我们对 `main.cpp` 文件进行编译得到 `main.exe`，接着以管理员身份运行该文件，未出现报错信息，得到运行过程如下：

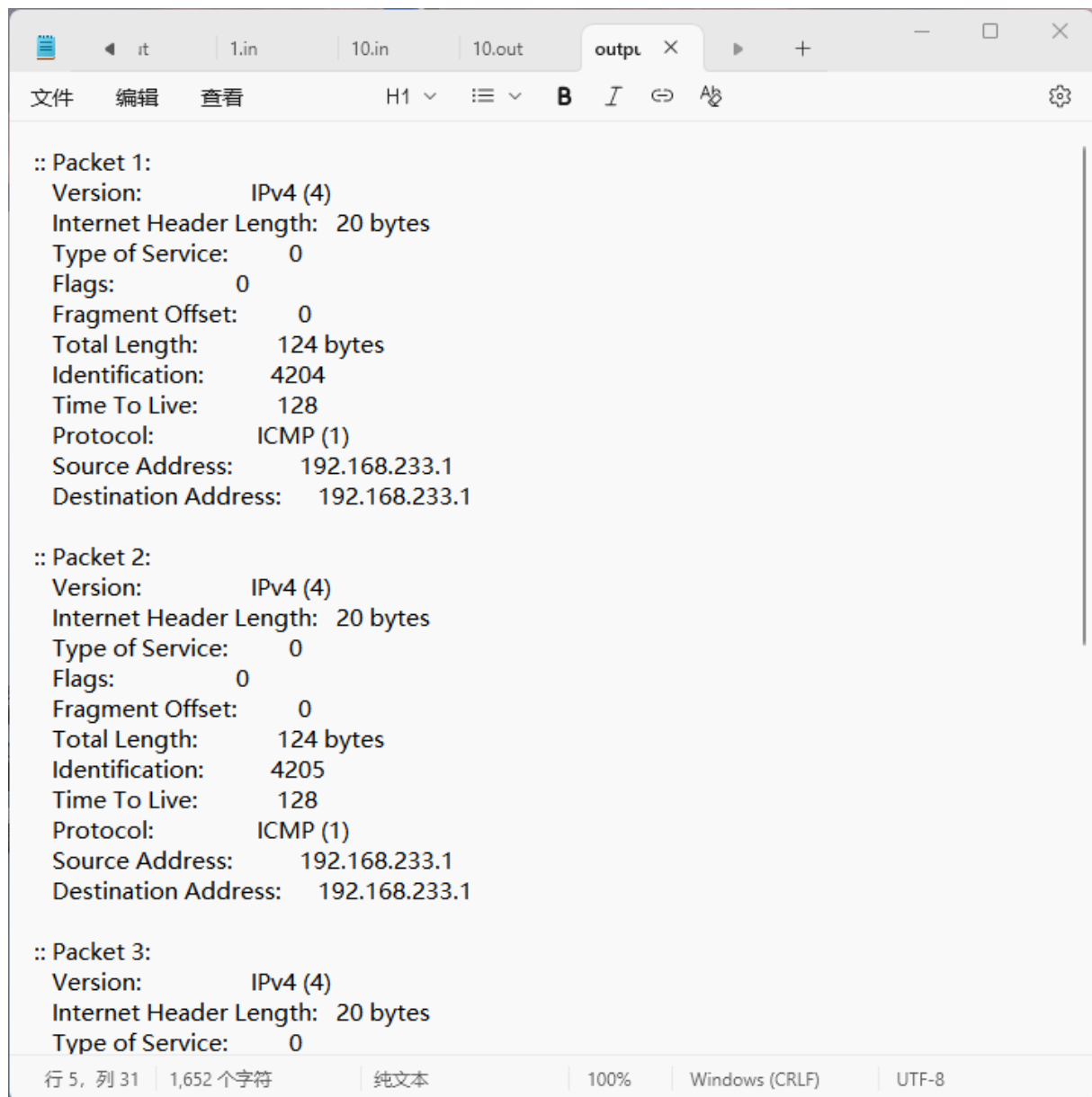


```
C:\Users\19711\Desktop\main.exe
The results will be output to the default file: output.txt
Hostname: LAPTOP-2M4NV27L
IP: 192.168.233.1
....
```

图 44: 抓包过程截图

每输出一个点表示成功抓到了一个包。当抓到的包数量足够多后，我们用 `Ctrl + C` 中断程序停止抓包。

我们可以看到当前目录下多了 `output.txt` 输出文件，点开里面是解析好的 IP 数据包信息。



```
:: Packet 1:
  Version:          IPv4 (4)
  Internet Header Length: 20 bytes
  Type of Service:   0
  Flags:             0
  Fragment Offset:   0
  Total Length:      124 bytes
  Identification:    4204
  Time To Live:      128
  Protocol:          ICMP (1)
  Source Address:    192.168.233.1
  Destination Address: 192.168.233.1

:: Packet 2:
  Version:          IPv4 (4)
  Internet Header Length: 20 bytes
  Type of Service:   0
  Flags:             0
  Fragment Offset:   0
  Total Length:      124 bytes
  Identification:    4205
  Time To Live:      128
  Protocol:          ICMP (1)
  Source Address:    192.168.233.1
  Destination Address: 192.168.233.1

:: Packet 3:
  Version:          IPv4 (4)
  Internet Header Length: 20 bytes
  Type of Service:   0
```

图 45: output.txt 文件

输出结果包含了版本号、总长度、标识、标志位、片偏移、协议、源地址和目的地址，符合实验要求。

2.3.8 实验结果分析

接下来我们节选一个输出片段进行分析：

```
:: Packet 1:
  Version:          IPv4 (4)
  Internet Header Length: 20 bytes
  Type of Service:   0
  Flags:             0
```

Fragment Offset:	0
Total Length:	124 bytes
Identification:	4204
Time To Live:	128
Protocol:	ICMP (1)
Source Address:	192.168.233.1
Destination Address:	192.168.233.1

分析:

- 抓到的数据报为 IPv4 数据报。
- 首部长度字段值为 5，对应 20 字节，说明该数据报未携带 IPv4 可选字段。
- 服务类型字段为 0，表示使用默认的服务质量。
- 标志位为 0，说明该数据报未被设置分片相关标志。
- 片偏移量为 0，表示该数据报未发生分片。
- 数据报总长度为 124 字节，包含 IPv4 首部与上层协议数据。
- 标识字段值为 4204，用于在发生分片时对数据报进行重组。
- 生存时间（TTL）为 128，说明该数据报最多可经过 128 跳路由器转发。
- 上层协议类型为 ICMP，对应协议号为 1。
- 源地址与目的地址均为 192.168.233.1，说明该 ICMP 数据报为主机本地发起并接收的数据包，常见于本地连通性测试。

2.3.9 程序源代码

下面给出本实验所实现的完整程序源代码，采用 C++ 语言编写，基于 Windows 平台的 raw socket 机制实现 IP 数据包的捕获与解析。

Listing 1: IP 数据包捕获与解析程序源码

```
1 #include <bits/stdc++.h>
2
3 // raw socket 编程所需头文件，需要链接到动态库 WS2_32
4 #include <winsock2.h>
5 #include <mstcpip.h>
6
7 const char *const filePath = "output.txt";
8 const int maxHostNameLen = 256;      // 8 位
```



```
9  const int bufferSize = 65536; // 16 位
10
11  char buffer[bufferSize];
12
13  struct IP_datagram {
14      unsigned char Version_IHL; // 版本 4 位 + 首部长 4 位
15      unsigned char TOS; // 服务类型
16      unsigned short TL; // 总长度
17      unsigned short id; // 标识
18      unsigned short flags_fragOffset; // 标志位 3 位 + 片偏移 13 位
19      unsigned char TTL; // 生存时间
20      unsigned char protocol; // 协议
21      unsigned short headerChecksum; // 首部校验和
22      unsigned int srcAdd; // 源地址
23      unsigned int destAdd; // 目的地址
24
25      std::string getProtocol() const {
26          switch (protocol) {
27              case 1: return "ICMP (1)";
28              case 2: return "IGMP (2)";
29              case 6: return "TCP (6)";
30              case 17: return "UDP (17)";
31              default: return std::to_string((int)protocol);
32          }
33      }
34
35      std::string getVersion() const {
36          unsigned char version = Version_IHL >> 4;
37          switch (version) {
38              case 4: return "IPv4 (4)";
39              case 6: return "IPv6 (6)";
40              default: return std::to_string((int)version);
41          }
42      }
43
44      std::string getFlag() const {
45          unsigned char flags = flags_fragOffset >> 13;
46          std::string description;
47          if (flags & 0b001) description += "more fragments, ";
48          if (flags & 0b010) description += "don't fragment, ";
49          if (description.empty()) return "0";
```

```
50     description.pop_back();
51     description.pop_back();
52     description += " (" + std::to_string(flags) + ")";
53     return description;
54 }
55
56 void invertByte() {
57     unsigned short inv[10];
58     unsigned short *p = (unsigned short *)this;
59     for (int i = 0; i < 10; i++) {
60         memcpy(inv + i, p, sizeof(unsigned short));
61         if (i == 1 || i == 2 || i == 3 || i == 5)
62             inv[i] = ntohs(inv[i]);
63         p++;
64     }
65     memcpy(this, inv, sizeof(unsigned short) * 10);
66 }
67 };
68
69 static int count = 0;
70
71 void analyze(char *buffer, int length) {
72     auto *ip = (IP_datagram *)buffer;
73     ip->invertByte();
74     SOCKADDR_IN address{};
75
76     printf(":: Packet %d:\n", count);
77     printf("    Version:                %s\n", ip->getVersion().
78         c_str());
79     printf("    Internet Header Length:    %d bytes\n",
80         (ip->Version_IHL & 0b00001111) * 4);
81     printf("    Type of Service:          %d\n", ip->TOS);
82     printf("    Flags:                    %s\n", ip->getFlag().c_str()
83         );
84     printf("    Fragment Offset:          %d\n",
85         ip->flags_fragOffset & 0b0001111111111111);
86     printf("    Total Length:             %d bytes\n", ip->TL);
87     printf("    Identification:          %d\n", ip->id);
88     printf("    Time To Live:             %d\n", ip->TTL);
89     printf("    Protocol:                 %s\n", ip->getProtocol().
90         c_str());
```

```
88
89     address.sin_addr.s_addr = ip->srcAddr;
90     printf("    Source Address:      %s\n",
91            inet_ntoa(address.sin_addr));
92     address.sin_addr.s_addr = ip->destAddr;
93     printf("    Destination Address:    %s\n",
94            inet_ntoa(address.sin_addr));
95
96     puts("");
97 }
98
99 int captureData() {
100     WSADATA wsa_data{};
101     char hostname[maxHostNameLen];
102
103     DWORD dwInBuff = 1;
104     DWORD dwOutBuff;
105     DWORD dwBytesRet;
106
107     if (WSAStartup(0x0202, &wsa_data)) {
108         fprintf(stderr, "WSA startup failed.\n");
109         return -1;
110     }
111
112     if (gethostname(hostname, maxHostNameLen)) {
113         fprintf(stderr, "Failed to get hostname.\n");
114         WSACleanup();
115         return -1;
116     }
117
118     HOSTENT *host = gethostbyname(hostname);
119
120     SOCKADDR_IN address{};
121     address.sin_addr.s_addr = *(unsigned long *)host->h_addr;
122     address.sin_family = AF_INET;
123     address.sin_port = htons(0);
124
125     SOCKET sockServer = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
126     if (sockServer < 0) {
127         WSACleanup();
128         return -1;
```

```
129     }
130
131     if (bind(sockServer, (SOCKADDR *)&address, sizeof(SOCKADDR))) {
132         closesocket(sockServer);
133         WSACleanup();
134         return -1;
135     }
136
137     if (WSAIoctl(sockServer, SIO_RCVALL,
138                 &dwInBuff, sizeof(dwInBuff),
139                 &dwOutBuff, sizeof(dwOutBuff),
140                 &dwBytesRet, nullptr, nullptr)) {
141         closesocket(sockServer);
142         WSACleanup();
143         return -1;
144     }
145
146     while (true) {
147         memset(buffer, 0x00, sizeof(buffer));
148         int bufferSize = recv(sockServer, buffer, maxBufferSize, 0);
149         if (bufferSize < 0) break;
150         ++count;
151         analyze(buffer, bufferSize);
152     }
153
154     closesocket(sockServer);
155     WSACleanup();
156     return 0;
157 }
158
159 int main(int argc, char *argv[]) {
160     if (argc == 1) {
161         freopen(filePath, "w", stdout);
162     } else if (argc == 2) {
163         freopen(argv[1], "w", stdout);
164     } else {
165         return -1;
166     }
167     return captureData();
168 }
```

三、 课程设计总结体会

通过本次计算机网络课程设计，我对课堂上所学习的网络基础知识有了更加系统和深入的认识。从最初对网络结构和设备功能理解不够清晰，到能够独立完成网络拓扑的搭建与配置，我逐步体会到了理论知识在实际应用中的重要性。在实验过程中，我熟练掌握了如 ping、ipconfig、tracert、show ip route 等常用网络命令的使用方法，并能够通过命令输出来分析网络的连通性和故障原因。同时，我对交换机和路由器的基本配置流程有了清晰的认识，包括设备初始化、接口配置、VLAN 划分以及路由设置等内容。

在综合网络情景设计部分，我将所学的网络原理与实际操作相结合，围绕多 VLAN 环境下的通信需求完成了整体网络的规划与实现。在查阅和整理大量相关资料的基础上，我成功配置了三层交换机与路由器的互联，并通过动态路由协议实现了各个网络之间的通信。此外，在内外网隔离的实验环境中，通过在出口路由器上配置 NAT 技术，使内网主机能够正常访问外网服务器，从而加深了我对地址转换原理及其应用场景的理解。在此过程中，我还完成了 WEB 和 DNS 服务器的基本配置，对服务器在网络中的作用和工作方式有了更加直观的认识。

在网络编程部分，我初步接触并学习了基于 Socket 的网络编程方法，通过编写程序实现了简单的数据抓包功能。通过对捕获到的数据进行分析，我能够将课堂上所讲的 IP 数据报结构、协议字段等知识与实际数据对应起来，加深了对网络数据传输过程的理解。这一过程不仅提升了我的动手能力，也让我体会到理论与实践相结合所带来的学习效果和成就感。

总体而言，本次课程设计内容完整、实践性强，使我在网络设备配置、综合网络设计以及网络编程等多个方面都得到了锻炼和提升。通过不断调试和排错，我的分析问题和解决问题的能力也得到了明显增强。这次课程设计让我受益匪浅，为今后参与实际项目开发以及进行复杂网络环境的配置和维护打下了良好的基础。